

VERO STATUS 4.3.0 - COMPLETE SOURCE FOR REBUILD

Jira ASPF-1578 · Quality AI Automation · a document only: nothing in it runs

What this is: the full source (29 text files) of "Vero Status", a small program that shows in the browser whether Vero is available and which model it uses. Python 3.8+ standard library only. The guide PDF is sent separately.

INSTRUCTIONS FOR THE AI ASSISTANT (Vero CLI or other) - recreate the project folder EXACTLY:

- 1. Create the folder vero-status in the location the user gives (default C:\Tools\).
- 2. Every file starts with a row "=== FILE: <path> (<n> lines) ===" and ends with "=== END FILE: <path> ===".
- 3. Rows between them encode the file, one source line per numbered row:
 - " 12 | content" -> line 12. Every `␣` (open box) in the content is ONE SPACE. It marks leading spaces, runs of 2+ spaces and spaces at a row edge; single spaces between words are printed normally. The `␣` character never occurs in the real files.
 - " 12 |" -> line 12 is an empty line.
 - " + more" -> continuation of the previous line: append "more" directly (no space, no newline). Ignore page headers/footers ("Vero Status ... source for rebuild", "page N").
- Line numbers must run 1..n without gaps; n is in the FILE row. Every file ends with a newline.
- 4. Write files as UTF-8 without BOM, LF line endings. Do not reformat, translate, summarise or "fix" anything.
- 5. Create verify_rebuild.py from the section VERIFY SCRIPT at the end (same row rules), OUTSIDE the project folder, and run: `py verify_rebuild.py C:\Tools\vero-status`
It checks every file's SHA-256 and sets CRLF line endings where Windows needs them (.cmd, START_HERE.txt). Recreate any file reported MISSING or DIFFERENT, then run it again until it prints "OK: all files match".
- 6. Run the tests inside the folder: `py -m unittest discover -s tests -t .` Expected: OK (54 tests).
- 7. Tell the user it is ready: double-click "Vero Status.pyw" to open the dashboard (see START_HERE.txt).
Do not start the program or call the real Vero CLI unless the user asks.

FILE LIST (path · lines · line endings · SHA-256 of the content with LF endings)

.gitattributes	7	LF	efa38e63b1747daed572feeff39747d40fd4cfa3a4568b11815cb0bf24332d96
.gitignore	8	LF	b592d17a523d7d8d8bd950419afdeb5f502d4aedd8b65a2887bd71f0c514aa33
CHANGELOG.md	49	LF	9e4bf2019070d5e3d68965891bc318e750e39894aaf938de4e57b6dc7dfb463
README.md	103	LF	d2d8209f7a4be482593f1c1e91dfb14d725429ef550a490344c5ea5b292d6de
START_HERE.txt	26	CRLF	3d6c38c0e04d3c5791716f49bab854b9ab72ed69d2824c3eccd6363e5c3682ba
VERO_CONTEXT.md	108	LF	ea8d9dd774e16d3a6b25c667d8ea6e32d42baf9c91566d30b28f02f7511719c0f
Vero Status.pyw	10	LF	d225f8171b6196c264964f66cba8c2992695a171db6c2f047f9c694fb8358197
pyproject.toml	23	LF	dcdab737202f10c2b1610fa9020563de6021071504855e756ae8d5467fd017e9
scripts/build_source_pdf.py	224	LF	fc3fa158ab8de632655566bc17697fcf54f028e743f0c56dccbf9e0bc4ccd690
scripts/fake_session.py	40	LF	5fa5b85d17b1e8ae07e72460a01f242d7e5223c55eb96f4d6d31fb8087a007a7
scripts/fake_vero.py	56	LF	4908eb33e4a64621f955e551ef0efce2e644ca3a6ba51e9c202b80cf01d35ae8
scripts/install_vm.sh	178	LF	23c89644435950587375e4e986ca43a702b9cbaf619849aa976e1832242e0b42
scripts/vero	2	LF	45890958a3fcde4e02f1d4c41412869e6f71a9efe2d2d237cd54cf8d84e001fa
scripts/vero.cmd	2	CRLF	15e9a293d205d911b2e2746af1c4361906c0b4138dc111be5acc2a653a6cd0ca
tests/__init__.py	1	LF	8d8ab1348161355b2c6856609575f5becac99c397071c7dcdad95d00597f39fb
tests/helpers.py	29	LF	1ee6057626c3d29cae5cca50a5a4583151c3c6d109d17927c066263c15523e03
tests/test_monitor_server.py	310	LF	60dabb972cad94f42b283bb9e7c0dc432882a58cc4644a9fde869e783c574a2a
tests/test_sessions.py	185	LF	86152ace8a67751301e0e94be87619f3133321a62be2381b911ec89b8e83fe36
tests/test_settings.py	46	LF	d5c501ab341e42d07021d55e29832d40a0780fa8bac8fae525af14359183b836
tests/test_vero.py	124	LF	972fb76ca969e62456c382dc55d47f9f25ad0e5c3ccf49f85b841bd7af93d65d
vero_status/__init__.py	4	LF	db5eb515f4f26125dc8f7a6a34341a0211ald05a20286dbdbd1885e198ed81
vero_status/app.py	184	LF	80ce46725991064987f6f5e98a35fa38e4effa1ecf40960ac6bc404cad814c6e
vero_status/monitor.py	230	LF	cd0dc809649408e8ce6e3eedea4aad60bcee9c61e9fcb1ae2935c4e99131da9
vero_status/server.py	140	LF	ef08ac947eff7830d8f8611c469d6b5e4c0281fb3a095cd2927931acf2a00348
vero_status/sessions.py	293	LF	837e129dc53707a16169ce28fc9ce6533db4a1d82bbeab75134e3fcf57d6f03a
vero_status/settings.py	65	LF	d362a34291c878bc40a6a3998a3a19633e9918aa1ac8d4352f0add2618c47d2d
vero_status/vero.py	223	LF	2a429351dc5a4b26352b4feb9c3dd6f0ba40f7e388c6cbf7b9e7ee6524ac82e4
vero_status/web/index.html	383	LF	355dd1b6a1b3b4bedb7c24b0c67f2541d800785b9ccaa103076832b8a0c0523c
vero_status_server.py	16	LF	e05c2030adc4991a41d93f95cefd1d5d6785623dc96bf9eb10c20bf4a44f05a0

```
=== FILE: .gitattributes (7 lines) ===
1 | * text=auto
2 | *.py text eol=lf
3 | *.pyw text eol=lf
4 | *.md text
5 | *.cmd text eol=crlf
6 | START_HERE.txt text eol=crlf
7 | scripts/vero text eol=lf
=== END FILE: .gitattributes ===
```

=== FILE: .gitignore (8 lines) ===

```
1 | __pycache__/  
2 | *.py[cod]  
3 | data/  
4 | dist/  
5 | .venv/  
6 | .mypy_cache/  
7 | .ruff_cache/  
8 | *.egg-info/
```

=== END FILE: .gitignore ===

=== FILE: CHANGELOG.md (49 lines) ===

```

1 | # Changelog
2 |
3 | ## 4.3.0 (Linux VM)
4 | - `scripts/install_vm.sh`: runs Vero Status in the background on a Linux VM and starts it with the machine
5 |   systemd user service, else crontab @reboot); `personal` (default) or `team`, plus `status` and `uninstall`. No sudo.
6 |   Copies PATH and proxy settings for the service into `data/service.env` (owner-only), never AWS keys.
7 | - Vero CLI found without PATH: nvm (newest node first), `~/.npm-global/bin`, `~/.local/bin`, `/usr/local/bin`.
8 |   The program's folder is put first on the child PATH so `#!/usr/bin/env node` finds nvm's node.
9 | - Headless: no browser is started without a desktop; the URL is printed and logged instead.
10 | - Port forwarding works: local mode accepts 127.0.0.1 / localhost on any port (VS Code may forward 8767 as 8768).
11 |   Before, a different forwarded port got 403.
12 | - Shared VM: `data/` is owner-only (0700, admin key 0600); only your own processes count as an open Vero CLI;
13 |   under systemd/cron the admin link goes to the private log only, not the system journal.
14 | - VS Code Remote-SSH session logs (`~/vscode-server/data/User/globalStorage`) are read.
15 | - SIGTERM (systemd stop, kill) stops cleanly. The page footer shows the machine the checks run on.
16 |
17 | ## 4.2.0
18 | - **Team server**: `python vero_status_server.py` serves one shared page on the network. No login or registration:
19 |   anyone with the link sees availability, models, 24 h availability and the new **Recent activity** table.
20 | - Viewers can trigger **Check now** at most once every 5 minutes; settings need the admin link
21 |   (`?admin=<key>`, key in `data/admin_key.txt`, compared in constant time, removed from the address bar).
22 | - On the server there is no Quit, the personal panels are off, and the Vero path is hidden from viewers.
23 |   Cross-site POSTs are still rejected.
24 | - New **Recent activity** panel (also locally): last 10 checks with result, model, answer time and trigger.
25 |
26 | ## 4.1.0
27 | - New panel **Vero open now**: when a Vero CLI is running, or a Vero CLI / VS Code session was active in the last
28 |   10 minutes, it shows the model in use right now (idle CLI: Vero's configured model). "Waiting for the model for N s"
29 |   while a request runs.
30 | - New panel **Your Vero sessions: response time**: for every model request in your own sessions (CLI and VS Code),
31 |   the time until the model started answering, read from Vero's own task logs (`ui_messages.json`). Median and p95 for
32 |   the last hour, median for 24 h, and the last 40 requests as bars with details on hover.
33 | - Privacy: only timestamps, event kinds, model names and token counts are read; prompts and answers never. Files are
34 |   only read. The dashboard's own test questions are excluded (by task id and prompt). Setting to switch it off.
35 | - Process detection is structural (vero / vero.cmd / node running a `vero` package), so Vero Status itself, admin
36 |   commands (`vero version`, `config`, ...) and its own checks are never mistaken for an open session.
37 |
38 | ## 4.0.0
39 | Rewritten as a simple program with a dashboard (feedback on 3.0: too many command windows and steps).
40 | - Double-click `Vero Status.pyw`: no console, no setup, no Task Scheduler. A second double-click just opens the dashboard.
41 | - The dashboard shows whether Vero is available and **which model Vero uses**: the configured model (`vero config`)
42 |   and the model asked in the last check (`modelInfo`). Also the Vero version, response time, 24 h availability and the last checks.
43 | - Check now, Settings (interval: off / 5 / 15 / 30 / 60 min, check model, timeout, Vero location) and Quit on the page.
44 | - Keeps the safe parts of 3.0: `--json` stream with `completion_result` only, no `--yolo`, empty working folder,
45 |   the whole process tree killed on timeout, Host and Origin checks, nothing secret stored. Errors at start show a Windows message box.
46 | - 30 tests (fake Vero), ruff, mypy strict for Linux and Windows, checked in a real browser.
47 |
48 | ## 3.0.0 and older
49 | Vero Availability Monitor (3.0) and Vero Latency Monitor (0.x): command-line tools with installer scripts. Superseded.

```

=== END FILE: CHANGELOG.md ===

=== FILE: README.md (103 lines) ===

```

1 | # Vero Status
2 |
3 | Double-click Vero Status and a dashboard opens in your browser showing:
4 |
5 | - Is Vero available? Big green or red status, since when, and the reason if not.
6 | - Which model does Vero use? The model Vero is configured with (`vero config`), and the model that answered the last check.
7 | - The Vero CLI version, how long the last answer took, availability over the last 24 h, and the last checks.
8 | - Vero open now: when you have a Vero CLI open, or are using Vero in VS Code, the model in use right now.
9 | - Your Vero sessions: response time: how long the model takes to start answering in your CLI and VS Code
10 | sessions (median and p95 for the last hour, 24 h median, last 40 requests as bars).
11 |
12 | Buttons: Check now, Settings (how often to check, which model to use for the check, Vero location), Quit.
13 |
14 | Jira ASPF-1578 · Version 4.3.0 · Python 3.8+ standard library only, no install, no command window.
15 |
16 | ## Use it
17 |
18 | 1. Put the folder somewhere fixed, e.g. `C:\Tools\vero-status`.
19 | 2. Double-click Vero Status.pyw. The dashboard opens at http://127.0.0.1:8767.
20 | 3. Optional: right-click `Vero Status.pyw` and choose Send to > Desktop (create shortcut).
21 | To start it with Windows, put that shortcut in the folder that opens with `Win+R` -> `shell:startup`.
22 |
23 | Double-clicking again while it runs just opens the dashboard. Quit stops it.
24 |
25 | ## Team server (one page for everyone, no login)
26 |
27 | Run it once on a server (or any always-on PC) and share one link. Everyone opens it in the browser, no login,
28 | no registration, nothing to install on their laptops. One check serves the whole team.
29 |
30 | ...
31 | python vero_status_server.py# port 8767, all network interfaces
32 | python vero_status_server.py --port 8080# another port
33 | ``
34 |
35 | It prints (and writes to `data/vero-status.log`):
36 |
37 | - Share link `http://<server>:8767/`: anyone can see whether Vero is available, its model, 24 h availability,
38 | and Recent activity (the last 10 checks: time, result, model, answer time, who started it).
39 | Viewers can press Check now at most once every 5 minutes (shared for everyone).
40 | - Admin link `http://<server>:8767/?admin=<key>`: also shows Settings. Keep it private. The key is in
41 | `data/admin_key.txt`; delete that file and restart to get a new one. The key is removed from the address bar after opening.
42 |
43 | There is no Quit button and the personal panels (Vero open now, your sessions) are off: they only make sense on
44 | your own laptop, so keep using `Vero Status.pyw` locally for those.
45 |
46 | Windows server, start with the machine: Task Scheduler > Create Task > Run whether user is logged on or not,
47 | trigger At startup, action `pythonw.exe` with arguments `vero_status_server.py` and Start in the folder.
48 | Run it as the functional account that is signed in to Vero (`vero auth`). Open the port once (admin PowerShell):
49 |
50 | ...
51 | New-NetFirewallRule -DisplayName "Vero Status" -Direction Inbound -LocalPort 8767 -Protocol TCP -Action Allow
52 | ...
53 |
54 | Linux: a systemd service with `ExecStart=/usr/bin/python3 /opt/vero-status/vero_status_server.py` and `Restart=always`.
55 |
56 | Plain HTTP is fine on the intranet (the page holds no secrets); put IIS or nginx in front for HTTPS.
57 | Record the host, the functional account and the link in the SCMP (`servers.md`, `functional-accounts.md`, `publishing.md`).
58 |
59 | ## On a Linux VM (no desktop)
60 |
61 | ...
62 | bash scripts/install_vm.sh# personal: 127.0.0.1, your own sessions; starts with the VM
63 | bash scripts/install_vm.sh team# team server on all interfaces (port must be open to the VM)
64 | bash scripts/install_vm.sh status# or: uninstall
65 | ``
66 |

```

```

67 | Open it from your laptop through VS Code port forwarding (PORTS tab, port 8767) or
68 | `ssh -N -L 8767:127.0.0.1:8767 <user>@<vm-host>`. It uses a systemd user service (or crontab @reboot), needs no sudo,
69 | and copies your PATH and proxy settings for the service. To keep it running after a reboot without a login, the VM admin
70 | runs `sudo loginctl enable-linger <user>` once. Unzip with `python3 -m zipfile -e <zip> .` if `unzip` is missing.
71 |
72 | ## How a check works
73 |
74 | 1. `vero version`: is Vero installed?
75 | 2. `vero config`: which model and provider Vero is set to (free).
76 | 3. `vero task --json -t 120 -c <empty folder> -m <check model> "Reply with exactly: OK"`.
77 | Vero is **available** when the `completion_result` answer arrives. This costs 2 small model calls and takes about 1 minute.
78 | The check model defaults to the cheap Haiku 4.5; leave it empty in Settings to test Vero's own model.
79 |
80 | The checks never use `--yolo`, run in an empty folder, and never store answers or secrets.
81 | The data lives in the program's `data/` folder (`settings.json`, `history.json`, `vero-status.log`).
82 |
83 | ## How your own sessions are measured
84 |
85 | Vero writes every task's events to `ui_messages.json`: for the CLI in `%USERPROFILE%\vero\data\tasks\<id>\`,
86 | for VS Code in the extension's storage (`%APPDATA%\Code\User\globalStorage\<vero extension>\tasks\<id>\`).
87 | Each model request starts with an `api_req_started` event; the next event is the model's first response.
88 | The difference is the response time, using Vero's own timestamps. A running `vero` / `node ...vero...` process
89 | means the CLI is open. Only timestamps, event kinds, model names and token counts are read, never your prompts
90 | or answers, and nothing is changed. The dashboard's own test questions are left out. Switch it off in Settings.
91 |
92 | ## Try it without Vero
93 |
94 | In Settings, set *Vero CLI location* to `scripts\vero.cmd` (Windows) or `scripts/vero` (Linux/macOS; run `chmod +x scripts/vero` once): a fake Vero that answers like the real one.
95 |
96 | ## Develop
97 |
98 | ```
99 | python -m unittest discover -s tests -t .# 54 tests, fake Vero only
100 | uvx ruff check . && uvx ruff format --check .# lint + format
101 | uvx mypy && uvx mypy --platform win32.# strict typing
102 | uv run --no-project --with reportlab python scripts/build_source_pdf.py.# source as PDF for rebuild
103 | ```
=== END FILE: README.md ===

```

```

=== FILE: START_HERE.txt (26 lines) ===
1 | VERO STATUS 4.3 (ASPF-1578)
2 | =====
3 |
4 | Shows in your browser whether Vero is available and which model it uses.
5 | Needs Python 3.8+ only. Nothing to install. No command window.
6 |
7 | 1. Put this folder somewhere fixed, e.g. C:\Tools\vero-status
8 |
9 | 2. Double-click "Vero Status.pyw"
10 | Your browser opens the dashboard (http://127.0.0.1:8767). The first check takes about a minute.
11 |
12 | 3. Optional: right-click "Vero Status.pyw" -> Send to -> Desktop (create shortcut).
13 | To start it with Windows: Win+R -> shell:startup -> put that shortcut there.
14 |
15 | In the dashboard: Check now | Settings (how often, which model, Vero location) | Quit
16 |
17 | If an older version is installed (Vero Latency Monitor or Vero Availability Monitor), run
18 | uninstall.bat in its folder so it stops calling Vero.
19 |
20 | TEAM SERVER (one shared page, no login): on an always-on machine run
21 | python vero_status_server.py
22 | and share the link it prints. See README.md, section Team server.
23 |
24 | LINUX VM (no desktop): bash scripts/install_vm.sh (see README.md, On a Linux VM)
25 |
26 | Problems: open data\vero-status.log, or read Vero_Status_Guide.pdf.
=== END FILE: START_HERE.txt ===

```

```
=== FILE: VERO_CONTEXT.md (108 lines) ===
```

```
1 | # VERO_CONTEXT: Vero Status 4.3.0
2 |
3 | > **For the AI reading this:** this file describes the whole program. Read a file before changing it, and keep the rules at the end.
4 |
5 | ## What it is
6 |
7 | A small program for one laptop. When you double-click `Vero Status.pyw`, it opens a dashboard in the browser at `http://127.0.0.1:8767`. The dashboard shows:
8 |
9 | - whether **Vero is available** (Vero CLI 2.3.x);
10 | - **which model Vero uses**: the configured model from `vero config`, and the model the last check used (`modelInfo` in the `--json` stream);
11 | - the Vero version, the response time, availability over 24 h, and the last 48 checks;
12 | - **Vero open now**: the model in use in your own open Vero CLI or VS Code session;
13 | - **Your Vero sessions: response time**: median and p95 per hour, the 24 h median, and the last 40 requests;
14 | - **Recent activity**: the last 10 checks (time, result, model, answer time, trigger);
15 | - buttons for Check now, Settings and Quit.
16 |
17 | **Team server** (4.2): `python vero_status_server.py [--port N] [--host H]` serves the same page on the network for
18 | everyone, with no login. Viewers see status, models, history and Recent activity; their Check now is limited to once per
19 | 5 minutes (`server.COOLDOWN_S`). Settings need the admin key (`?admin=<key>` in the URL, sent as the `X-Admin-Key` header,
20 | stored in `data/admin_key.txt`, compared with `hmac.compare_digest`). No Quit endpoint, no personal session panels,
21 | and the Vero path is hidden from viewers.
22 |
23 | It uses only the Python 3.8+ standard library. It opens no console window (`pythonw`), and every child process starts with `CREATE_NO_WINDOW`.
24 |
25 | The Jira story is ASPF-1578, for the Quality AI Automation team at NXP. Version 4.0.0 replaces the 3.0 "Vero Availability Monitor", which was command-line based.
26 |
27 | ## Files
28 |
29 | ```
30 | Vero Status.pyw: double-click entry: adds the folder to sys.path, calls vero_status.app.run()
31 | vero_status_server.py: team server entry: calls vero_status.app.serve_team() (admin key, binds 0.0.0.0)
32 | vero_status/
33 |   __init__.py: version = "4.3.0", APP_ID = "vero-status"
34 |   app.py: single instance (GET /api/health), logging to data/vero-status.log, opens the browser,
35 |   shows a Windows message box on errors (no console)
36 |   vero.py: find_vero, run (no window, kills the process tree on timeout), and pure parsers:
37 |   parse_version, parse_config, parse_task_stream, task_args, clean (masks credentials)
38 |   sessions.py: your own Vero sessions: parse_ui_messages (pure: request latency = first event after
39 |   api_req_started; model from modelInfo), Scanner (finds tasks/*/ui_messages.json for CLI
40 |   and VS Code, cached by mtime, every 15 s), is_interactive_vero / process_command_lines
41 |   (a vero / vero.cmd / node-with-vero-package process = an open CLI), summarize
42 |   monitor.py: perform_check(settings, workdir, run) -> record; Monitor: timer thread, check_now,
43 |   history (data/history.json, last 500), update_settings, state() for the page
44 |   settings.py: Settings (frozen dataclass), validation, load/save of data/settings.json
45 |   server.py: GET /, /api/state, /api/health; POST /api/check, /api/settings, /api/quit (local only).
46 |   Local: 127.0.0.1, Host and Origin checked. Server mode: any Host, POST Origin checked,
47 |   settings need the admin key, viewer Check now cooldown (429).
48 |   web/index.html: the dashboard (one file, vanilla JS, light and dark, polls /api/state)
49 |   scripts/fake_vero.py: stand-in Vero: version, config, task --json (FAKE_VERO_MODE ok|down|auth|hang|nocompletion|refuse)
50 |   scripts/fake_session.py: writes a realistic session log into a folder (demo and tests; never your real ~/.vero)
51 |   scripts/vero, vero.cmd: launch the fake Vero (point "Vero CLI location" at one of these for a demo)
52 |   scripts/install_vm.sh: Linux VM: systemd user service or crontab; personal | team | status | uninstall
53 |   scripts/build_source_pdf.py: writes the source as a PDF for rebuilding (developer tool, needs reportlab)
54 |   tests/: unittest; never calls the real Vero
55 | ```
56 |
57 | ## Linux VM (4.3)
58 |
59 | Headless: `app.can_open_browser()` is false without DISPLAY/WAYLAND_DISPLAY, so the URL is printed. `vero.npm_locations()`
60 | finds nvm / npm-global installs and `vero.child_env()` puts the program's folder first on PATH (node for nvm).
61 | Local mode accepts loopback host names on any port (forwarded ports). `data/` is 0700, the admin key 0600.
62 | `process_command_lines()` lists only the current user's processes on POSIX. SIGTERM stops like Ctrl+C.
63 |
64 | ## One check (`monitor.perform_check`)
65 |
66 | 1. `find_vero(settings.vero_path)` looks for the Vero CLI in this order: the configured path, `vero` on the PATH (`vero.cmd` on Windows), then `%APPDATA%\npm\vero.cmd`. If nothing is found, the r
```

```

+ result is "not available" with the reason "not found".
67 | 2. `vero version`: if this fails, the result is "not available".
68 | 3. `vero config` reads the settings `actModeApiModelId`, `actModeApiProvider` and `awsRegion`, given either as `key : value` lines or as JSON. If this step fails, the configured model is simply u
+ nknown; it does not affect the status.
69 | 4. `vero task --json -t <timeout> -c data/sandbox [-m <check_model>] "Reply with exactly: OK"`, killed at timeout + 30 s.
70 | --- **Available** if and only if a `completion_result` event arrives. A different answer still counts as available, with a note.
71 | --- `partial` events are ignored. `api_req_started`, which holds the full prompt, is never kept.
72 | --- The record holds `time`, `available`, `reason`, `seconds` (only when there is an answer), `asked_model`, `provider`, `vero_version`, `configured` `{model}`, `provider`, `region`} and `trigger`
+ `(start, auto or manual)`.
73 |
74 | ## Settings
75 |
76 | Stored in `data/settings.json` and editable on the page:
77 |
78 | | Setting | Default | Allowed values |
79 | |---|---|---|
80 | | `vero_path` | `""` (find it automatically) | any path |
81 | | `check_model` | Haiku 4.5 | a model id, or `""` to use Vero's own model |
82 | | `interval_minutes` | 15 | 0 (off), 5, 15, 30, 60 |
83 | | `timeout_seconds` | 120 | 30-600 |
84 | | `port` | 8767 | 1024-65535; change it in the file only |
85 | | `track_sessions` | true | show your own sessions' model and response time |
86 |
87 | A broken settings file falls back to the defaults.
88 |
89 | ## Quality gates
90 |
91 | ```
92 | python -m unittest discover -s tests -t .(54 tests, Python 3.8-3.13)
93 | uvx ruff check . && uvx ruff format --check .
94 | uvx mypy && uvx mypy --platform win32(strict)
95 | ```
96 |
97 | ## Rules
98 |
99 | 1. Keep it simple for the user: double-click, a dashboard, no command windows, no installer.
100 | 2. Standard library only. No CDN and no external scripts in the page.
101 | 3. Never pass `--yolo`. Run the task in the empty `data/sandbox` folder. On timeout, kill the whole process tree.
102 | 4. Store no answers, no secrets and no `api_req_started` content. Mask credentials in error text.
103 | --- From the user's task logs, read only `ts`, event kinds, `modelInfo` and token counts. Only read those files, never change them.
104 | --- Exclude our own probe tasks, by task id and by prompt.
105 | 5. Local mode binds to 127.0.0.1 with Host and Origin checks. Server mode is opt-in (`vero_status_server.py`): keep the
106 | --- Origin check on POSTs, the admin key for settings, the viewer cooldown, and never expose paths or personal sessions.
107 | 6. Tests use the fake Vero only.
108 | 7. Commit messages and PR titles have the form `ASPF-1578: <summary>`. Update the CHANGELOG and this file when behaviour changes.
=== END FILE: VERO_CONTEXT.md ===

```

```
=== FILE: Vero Status.pyw (10 lines) ===
1 | """Double-click to open the Vero Status dashboard (no console window)."""
2 |
3 | import sys
4 | from pathlib import Path
5 |
6 | sys.path.insert(0, str(Path(__file__).resolve().parent))
7 |
8 | from vero_status.app import run_# noqa: E402
9 |
10 | run()
=== END FILE: Vero Status.pyw ===
```

```
=== FILE: pyproject.toml (23 lines) ===
```

```

1 | [project]
2 | name = "vero-status"
3 | version = "4.3.0"
4 | description = "Dashboard: is Vero available, and which model does it use? (ASPF-1578)"
5 | requires-python = ">=3.8"
6 | dependencies = []
7 |
8 | [tool.ruff]
9 | line-length = 130
10 | target-version = "py38"
11 |
12 | [tool.ruff.lint]
13 | select = ["E", "F", "W", "I", "B", "UP", "S", "SIM", "RUF"]
14 | ignore = ["S603", "S607", "S310", "UP006", "UP007", "UP035", "UP045", "RUF001", "RUF003"]
15 |
16 | [tool.ruff.lint.per-file-ignores]
17 | "tests/**" = ["S101", "S108"]
18 | "scripts/**" = ["S311"] ## demo randomness, not security
19 |
20 | [tool.mypy]
21 | python_version = "3.9"
22 | files = ["vero_status"]
23 | strict = true

```

```
=== END FILE: pyproject.toml ===
```

```
=== FILE: scripts/build_source_pdf.py (224 lines) ===
```

```

1 | #!/usr/bin/env python3
2 | """Build dist/Vero_Status_<version>_Source.pdf: the complete source as a PDF document.
3 |
4 | For channels that accept only documents. Developer tool (needs reportlab):
5 | uv run --no-project --with reportlab python scripts/build_source_pdf.py
6 |
7 | Listing format (survives PDF text extraction, which drops blank lines and collapses spaces):
8 | """
9 | line 12 | content"
10 | line 12 of the file; every MARK (U+2423, open box) in content is one space
11 | (used for leading spaces, runs of 2+ spaces and spaces at a row edge)
12 | line 12 | "
13 | line 12 is empty
14 | "
15 | + more"
16 | continuation of the previous line (joined with nothing in between)
17 | The MARK character never appears in the sources (checked at build time).
18 | """
19 |
20 | import hashlib
21 | import re
22 | import sys
23 | from pathlib import Path
24 |
25 | from reportlab.lib.pagesizes import A4, landscape
26 | from reportlab.pdfbase import pdfmetrics
27 | from reportlab.pdfbase.ttfonts import TTFont
28 | from reportlab.pdfgen import canvas
29 |
30 | ROOT = Path(__file__).resolve().parents[1]
31 | sys.path.insert(0, str(ROOT))
32 | from vero_status import __version__# noqa: E402
33 |
34 | SKIP_DIRS = {
35 |     "data",
36 |     "data-demo",
37 |     "reports",
38 |     "dist",
39 |     "__pycache__",
40 |     ".pytest_cache",
41 |     ".mypy_cache",
42 |     ".ruff_cache",
43 |     ".venv",
44 |     "venv",
45 | }
46 |
47 | SKIP_SUFFIXES = (".pyc", ".log", ".egg-info")
48 | SKIP_FILES = {"docs/Vero_Status_Guide.html", "docs/Vero_Status_Guide.pdf"}# images; shipped separately
49 | CRLF = (".bat", ".cmd")
50 | CRLF_FILES = {"START_HERE.txt"}
51 | FONT_PATHS = ["/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf", "C:/Windows/Fonts/consola.ttf"]
52 | SIZE, LEADING, MARGIN = 6.4, 7.7, 28
53 | PAGE_W, PAGE_H = landscape(A4)
54 | MARK = "\u2423"# open box, shown for a space that extraction could lose
55 |
56 | def files():
57 |     for f in sorted(ROOT.rglob("*")):
58 |         rel = f.relative_to(ROOT)
59 |         if (
60 |             f.is_dir()
61 |             or SKIP_DIRS & set(rel.parts)
62 |             or any(part.endswith(SKIP_SUFFIXES) for part in rel.parts)
63 |             or rel.as_posix() in SKIP_FILES
64 |         ):
65 |             continue
66 |         yield rel.as_posix(), f.read_bytes().decode("utf-8").replace("\r\n", "\n")
67 |
68 | def lf_sha(text: str) -> str:
69 |     return hashlib.sha256(text.encode("utf-8")).hexdigest()

```

```

67 |
68 | def protect(chunk: str) -> str:
69 |     """Mark every space an extractor could drop: leading, trailing, and runs of 2+."""
70 |     chunk = re.sub(r" {2,}", lambda m: MARK * len(m.group()), chunk)
71 |     if chunk.startswith(" "):
72 |         chunk = MARK + chunk[1:]
73 |     if chunk.endswith(" "):
74 |         chunk = chunk[:-1] + MARK
75 |     return chunk
76 |
77 |
78 | def encode(lines, width):
79 |     """Source lines -> listing rows (see module docstring)."""
80 |     rows = []
81 |     for n, line in enumerate(lines, 1):
82 |         head = f"{n:>4} |"
83 |         if not line:
84 |             rows.append(head)
85 |             continue
86 |         chunks = [line[i : i + width] for i in range(0, len(line), width)]
87 |         rows.append(f"{head} {protect(chunks[0])}")
88 |         rows.extend(f"      {protect(c)}" for c in chunks[1:])
89 |     return rows
90 |
91 |
92 | VERIFY_TEMPLATE = """import hashlib, pathlib, sys
93 | # usage: py verify_rebuild.py C:\\Tools\\vero-status
94 | root = pathlib.Path(sys.argv[1] if len(sys.argv) > 1 else "vero-status")
95 | CRLF = {crlf!r}
96 | FILES = {{
97 |     {entries}}}
98 | bad = 0
99 | for path, sha in FILES.items():
100 |     p = root / path
101 |     if not p.exists():
102 |         print("MISSING", path); bad += 1; continue
103 |     text = p.read_bytes().decode("utf-8").replace("\\r\\n", "\\n")
104 |     if hashlib.sha256(text.encode("utf-8")).hexdigest() != sha:
105 |         print("DIFFERENT", path); bad += 1; continue
106 |     want = text.replace("\\n", "\\r\\n") if path in CRLF else text
107 |     if p.read_bytes() != want.encode("utf-8"):
108 |         p.write_bytes(want.encode("utf-8")); print("fixed line endings", path)
109 |     print("OK: all files match" if not bad else str(bad) + " file(s) wrong: recreate them from the document")
110 |     sys.exit(1 if bad else 0)
111 | """
112 |
113 |
114 | class Doc:
115 |     def __init__(self, path: Path, font: str):
116 |         self.c = canvas.Canvas(str(path), pagesize=(PAGE_W, PAGE_H))
117 |         self.c.setTitle(f"Vero Status {__version__} - complete source for rebuild")
118 |         self.c.setAuthor("Quality AI Automation (ASPF-1578)")
119 |         self.c.setSubject("Source listing. A document only: nothing in it runs.")
120 |         self.font = font
121 |         self.page = 0
122 |         self.y = 0.0
123 |         self.new_page()
124 |
125 |     def new_page(self):
126 |         if self.page:
127 |             self.c.showPage()
128 |             self.page += 1
129 |             self.c.setFont(self.font, 6)
130 |             self.c.setFillGray(0.45)
131 |             self.c.drawString(MARGIN, PAGE_H - 16, f"Vero Status {__version__} · source for rebuild · ASPF-1578")
132 |             self.c.drawRightString(PAGE_W - MARGIN, 12, f"page {self.page}")
133 |             self.c.setFillGray(0)

```

```

134 |         self.y = PAGE_H - MARGIN - 8
135 |
136 |     def row(self, text: str, size: float = SIZE, gray: float = 0):
137 |         if self.y < MARGIN:
138 |             self.new_page()
139 |             self.c.setFont(self.font, size)
140 |             self.c.setFillGray(gray)
141 |             self.c.drawString(MARGIN, self.y, text)
142 |             self.y -= max(LEADING, size * 1.25)
143 |
144 |     def gap(self, n: float = 1):
145 |         self.y -= LEADING * n
146 |
147 |     def save(self):
148 |         self.c.save()
149 |
150 |
151 | def main() -> None:
152 |     font_path = next((p for p in FONT_PATHS if Path(p).exists()), None)
153 |     if not font_path:
154 |         sys.exit("no monospace TTF font found (DejaVu Sans Mono or Consolas)")
155 |     pdfmetrics.registerFont(TTFont("Mono", font_path))
156 |     char_w = pdfmetrics.stringWidth("M", "Mono", SIZE)
157 |     width = int((PAGE_W - 2 * MARGIN) / char_w) - 8 # minus the "NNNN | " gutter
158 |     items = list(files())
159 |     bad = [p for p, t in items if MARK in t]
160 |     if bad:
161 |         sys.exit(f"MARK character found in {bad}: the listing would be ambiguous")
162 |     crlf = sorted(p for p, _ in items if p.endswith(CRLF) or p in CRLF_FILES)
163 |     out = ROOT / "dist" / f"Vero_Status_{__version__}_Source.pdf"
164 |     out.parent.mkdir(exist_ok=True)
165 |     d = Doc(out, "Mono")
166 |
167 |     d.row(f"VERO STATUS {__version__} - COMPLETE SOURCE FOR REBUILD", 13)
168 |     d.row("Jira ASPF-1578 · Quality AI Automation · a document only: nothing in it runs", 8, 0.35)
169 |     d.gap()
170 |     intro = f"""\
171 | What this is: the full source ({len(items)} text files) of "Vero Status", a small program that shows in the browser whether
172 | Vero is available and which model it uses. Python 3.8+ standard library only. The guide PDF is sent separately.
173 |
174 | INSTRUCTIONS FOR THE AI ASSISTANT (Vero CLI or other) - recreate the project folder EXACTLY:
175 | 1. Create the folder vero-status in the location the user gives (default C:\\Tools\\).
176 | 2. Every file starts with a row{u}""== FILE: <path> (<n> lines) ""== and ends with{u}""== END FILE: <path> ""==.
177 | 3. Rows between them encode the file, one source line per numbered row:
178 | {u}12 | content"{u}-> line 12. Every {MARK} (open box) in the content is ONE SPACE. It marks leading
179 | {u}spaces, runs of 2+ spaces and spaces at a row edge; single spaces between words
180 | {u}are printed normally. The {MARK} character never occurs in the real files.
181 | {u}12 | ""{u}-> line 12 is an empty line.
182 | {u}+ more"{u}-> continuation of the previous line: append "more" directly (no space, no newline).
183 | {u}Ignore page headers/footers ("Vero Status ... source for rebuild", "page N").
184 | {u}Line numbers must run 1..n without gaps; n is in the FILE row. Every file ends with a newline.
185 | 4. Write files as UTF-8 without BOM, LF line endings. Do not reformat, translate, summarise or "fix" anything.
186 | 5. Create verify_rebuild.py from the section VERIFY SCRIPT at the end (same row rules), OUTSIDE the project
187 | folder, and run: {u}py verify_rebuild.py C:\\Tools\\vero-status
188 | {u}It checks every file's SHA-256 and sets CRLF line endings where Windows needs them (.cmd, START_HERE.txt).
189 | {u}Recreate any file reported MISSING or DIFFERENT, then run it again until it prints "OK: all files match".
190 | 6. Run the tests inside the folder: {u}py -m unittest discover -s tests -t {u}Expected: OK (54 tests).
191 | 7. Tell the user it is ready: double-click "Vero Status.pyw" to open the dashboard (see START_HERE.txt).
192 | Do not start the program or call the real Vero CLI unless the user asks.
193 | """
194 |     for line in intro.splitlines():
195 |         d.row(line, 7.2)
196 |         d.gap()
197 |     d.row("FILE LIST (path · lines · line endings · SHA-256 of the content with LF endings)", 7.2)
198 |     for p, t in items:
199 |         lines = t.split("\n")[:-1] if t.endswith("\n") else t.split("\n")
200 |         d.row(f'{u}{p[:52]} {len(lines):>5}{u}' 'CRLF' if p in crlf else 'LF{u}'{lf_sha(t)}', 6.2)

```

```

201 |
202 |     for p, t in items:
203 |         lines = t.split("\n")[:-1] if t.endswith("\n") else t.split("\n")
204 |         d.new_page()
205 |         d.row(f"=== FILE: {p} ({len(lines)} lines) ===", 7.5)
206 |         for r in encode(lines, width):
207 |             d.row(r)
208 |         d.row(f"=== END FILE: {p} ===", 7.5)
209 |
210 |     entries = "".join(f"{p!r}: {lf_sha(t)!r},\n" for p, t in items)
211 |     verify = VERIFY_TEMPLATE.format(crlf=crlf, entries=entries)
212 |     d.new_page()
213 |     vlines = verify.split("\n")[:-1]
214 |     d.row(f"=== VERIFY SCRIPT: verify_rebuild.py ({len(vlines)} lines) ===", 7.5)
215 |     for r in encode(vlines, width):
216 |         d.row(r)
217 |     d.row("=== END VERIFY SCRIPT ===", 7.5)
218 |     d.save()
219 |     (ROOT / "dist" / "verify_rebuild.py").write_text(verify, encoding="utf-8")
220 |     print(f"wrote {out} ({len(items)} files, {d.page} pages, {out.stat().st_size // 1024} KB, {width} chars per row)")
221 |
222 |
223 | if __name__ == "__main__":
224 |     main()
=== END FILE: scripts/build_source_pdf.py ===

```

```
=== FILE: scripts/fake_session.py (40 lines) ===
```

```

1 | #!/usr/bin/env python3
2 | """Simulate a user's Vero session for demos and tests: appends events to <tasks-dir>/<id>/ui_messages.json
3 | in the same shape Vero writes them (Cline-style events with ts, say, modelInfo).
4 |
5 | usage: python scripts/fake_session.py <tasks-dir> [--requests 6] [--model us.anthropic.claude-opus-5]
6 | Never point it at your real ~/.vero folder.
7 | """
8 |
9 | import argparse
10 | import json
11 | import random
12 | import time
13 | from pathlib import Path
14 |
15 | a = argparse.ArgumentParser()
16 | a.add_argument("tasks_dir")
17 | a.add_argument("--requests", type=int, default=6)
18 | a.add_argument("--model", default="us.anthropic.claude-opus-5")
19 | a.add_argument("--pause", type=float, default=1.5, help="seconds between requests")
20 | args = a.parse_args()
21 |
22 | info = {"providerId": "bedrock", "modelId": args.model, "mode": "act"}
23 | task = Path(args.tasks_dir) / f"demo-{int(time.time()) * 1000}"
24 | task.mkdir(parents=True, exist_ok=True)
25 | events = []
26 |
27 |
28 | def add(ev):
29 |     events.append({"ts": int(time.time()) * 1000, **ev, "modelInfo": info})
30 |     (task / "ui_messages.json").write_text(json.dumps(events), encoding="utf-8")
31 |
32 |
33 | add({"type": "say", "say": "task", "text": "demo task"})
34 | for _ in range(args.requests):
35 |     add({"type": "say", "say": "api_req_started", "text": json.dumps({"tokensIn": random.randint(800, 9000), "tokensOut": 0})})
36 |     time.sleep(random.uniform(1.5, 6.0))
37 |     add({"type": "say", "say": "text", "text": "...", "partial": True})
38 |     time.sleep(args.pause)
39 |     add({"type": "say", "say": "completion_result", "text": "done"})
40 |     print(f"wrote {task}")

```

```
=== END FILE: scripts/fake_session.py ===
```

```
=== FILE: scripts/fake_vero.py (56 lines) ===
```

```

1 | #!/usr/bin/env python3
2 | """Stand-in for the Vero CLI (demo and tests): `version`, `config`, `task --json ...` like Vero CLI 2.3.x.
3 |
4 | FAKE_VERO_MODE: ok (default) | down | auth | hang | nocompletion | refuse
5 | FAKE_VERO_DELAY: seconds before answering (default 2)
6 | """
7 |
8 | import json
9 | import os
10 | import sys
11 | import time
12 |
13 |
14 | def emit(obj):
15 |     print(json.dumps(obj), flush=True)
16 |
17 |
18 | def main(argv):
19 |     cmd = argv[0] if argv else "version"
20 |     if cmd in ("version", "--version"):
21 |         print("vero 2.3.3 (fake0000)")
22 |         return 0
23 |     if cmd == "config":
24 |         print("actModeApiProvider : bedrock")
25 |         print("actModeApiModelId: us.anthropic.claude-opus-5")
26 |         print("awsRegion: us-west-2")
27 |         return 0
28 |     if cmd not in ("task", "t"):
29 |         print(f"unknown command {cmd}", file=sys.stderr)
30 |         return 2
31 |     args = argv[1:]
32 |     model = args[args.index("-m") + 1] if "-m" in args else "us.anthropic.claude-opus-5"
33 |     info = {"providerId": "bedrock", "modelId": model, "mode": "act"}
34 |     mode = os.environ.get("FAKE_VERO_MODE", "ok")
35 |     delay = float(os.environ.get("FAKE_VERO_DELAY", "2"))
36 |     emit({"type": "task_started", "taskId": str(int(time.time() * 1000))})
37 |     emit({"type": "say", "say": "task", "text": args[-1], "modelInfo": info})
38 |     emit({"type": "say", "say": "api_req_started", "text": "SYSTEM PROMPT FAKE-SECRET-PROMPT", "modelInfo": info})
39 |     if mode == "hang":
40 |         time.sleep(3600)
41 |     if mode == "auth":
42 |         print("error: Unauthorized token=abc123secret", file=sys.stderr)
43 |         return 1
44 |     if mode == "down":
45 |         print("error: provider unavailable (connect ETIMEDOUT)", file=sys.stderr)
46 |         return 1
47 |     time.sleep(delay)
48 |     answer = "I cannot do that." if mode == "refuse" else "OK"
49 |     emit({"type": "say", "say": "text", "text": answer[1:], "partial": True, "modelInfo": info})
50 |     if mode != "nocompletion":
51 |         emit({"type": "say", "say": "completion_result", "text": answer, "modelInfo": info})
52 |     return 0
53 |
54 |
55 | if __name__ == "__main__":
56 |     sys.exit(main(sys.argv[1:]))
=== END FILE: scripts/fake_vero.py ===
```

```
=== FILE: scripts/install_vm.sh (178 lines) ===
```

```

1 | #!/usr/bin/env bash
2 | # Vero Status on a Linux VM (no desktop): run it in the background and start it with the machine.
3 | #
4 | #_scripts/install_vm.sh_ personal dashboard (127.0.0.1, with your own Vero sessions)
5 | #_scripts/install_vm.sh team [PORT]_ team server for colleagues (all interfaces, no login)
6 | #_scripts/install_vm.sh status_ is it running, and where
7 | #_scripts/install_vm.sh uninstall_ stop it and remove the autostart
8 | #
9 | # Uses a systemd user service when available, otherwise a crontab @reboot line. Needs no sudo.
10 | # Open the page from your laptop with VS Code port forwarding (PORTS tab) or an SSH tunnel.
11 | set -euo pipefail
12 |
13 | DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
14 | UNIT_DIR="${XDG_CONFIG_HOME:-$HOME/.config}/systemd/user"
15 | UNIT="$UNIT_DIR/vero-status.service"
16 | ENV_FILE="$DIR/data/service.env"
17 | TAG="# vero-status autostart"
18 | ACTION="{1:-personal}"
19 | USER="${USER:-$(id -un)}"
20 |
21 | say() { printf '%s\n' "$*"; }
22 | die() { printf 'ERROR: %s\n' "$*" >&2; exit 1; }
23 | have_systemd() { command -v systemctl >/dev/null && systemctl --user show-environment >/dev/null 2>&1; }
24 |
25 | python_bin() {
26 |     local py
27 |     for py in python3 python; do
28 |         if command -v "$py" >/dev/null && "$py" -c 'import sys; sys.exit(sys.version_info < (3, 8))' 2>/dev/null; then
29 |             command -v "$py"; return 0
30 |         fi
31 |     done
32 |     return 1
33 | }
34 |
35 | stop_background() {_# processes started by the crontab fallback
36 |     _kill -f "$DIR/Vero Status.pyw" 2>/dev/null || true
37 |     _kill -f "$DIR/vero_status_server.py" 2>/dev/null || true
38 | }
39 |
40 | uninstall() {
41 |     if [ -f "$UNIT" ]; then
42 |         _systemctl --user disable --now vero-status.service 2>/dev/null || true
43 |         _rm -f "$UNIT"
44 |         _systemctl --user daemon-reload 2>/dev/null || true
45 |         _say "Removed the systemd user service."
46 |         _fi
47 |         if command -v crontab >/dev/null && crontab -l 2>/dev/null | grep -qF "$TAG"; then
48 |             _crontab -l 2>/dev/null | grep -vF "$TAG" | crontab -
49 |             _say "Removed the crontab line."
50 |             _fi
51 |             _stop_background
52 |             _say "Vero Status is stopped. Your data/ folder (history, settings) is kept; delete the folder to remove everything."
53 |         }
54 |
55 | write_env() {_# what the service needs from your shell: PATH (nvm, npm) and the proxy. Never AWS keys.
56 |     _mkdir -p "$DIR/data"
57 |     _chmod 700 "$DIR/data"
58 |     local name value
59 |     _ : > "$ENV_FILE"
60 |     _chmod 600 "$ENV_FILE" _# a proxy URL can contain a password
61 |     for name in PATH HTTPS_PROXY HTTP_PROXY NO_PROXY https_proxy http_proxy no_proxy AWS_PROFILE AWS_REGION; do
62 |         value="{!name:-}"
63 |         _[ -n "$value" ] || continue
64 |         value="{value/\\/\\/\\/\\/}"
65 |         value="{value/\\/\\/\\/\\/}"
66 |         _printf '%s="%s"\n' "$name" "$value" >> "$ENV_FILE"

```

```

67 | _done
68 | }
69 |
70 | port_of() {_# personal: port from data/settings.json (default 8767)
71 | _ (cd "$DIR" && "$PY" -c 'from pathlib import Path; from vero_status.settings import load; print(load(Path("data/settings.json")).port)')
72 | }
73 |
74 | wait_healthy() {
75 |
76 | _for _ in $(seq 1 20); do
77 | _ _if "$PY" -c 'import sys, urllib.request; urllib.request.urlopen(f"http://127.0.0.1:{sys.argv[1]}/api/health", timeout=2)' "$1" 2>/dev/null; then
78 | _ _ _return 0
79 | _ _ _fi
80 | _ _ _sleep 1
81 | _ _done
82 | _ _return 1
83 | }
84 |
85 | status() {
86 | _PY="$(python_bin)" || die "Python 3.8+ not found"
87 | _local port="${1:-$(port_of)}"
88 | _if [ -f "$UNIT" ] && have_systemd; then
89 | _ _systemctl --user --no-pager status vero-status.service | head -n 5 || true
90 | _ _fi
91 | _if wait_healthy "$port"; then say "Running on port $port."; else say "Not answering on port $port."; fi
92 | }
93 |
94 | case "$ACTION" in
95 | _uninstall) uninstall; exit 0 ;;
96 | _status) status "${2:-}"; exit 0 ;;
97 | _personal | team) ;;
98 | _*) die "unknown action '$ACTION' (use: personal, team [PORT], status, uninstall)" ;;
99 | esac
100 |
101 | case "$DIR" in *) *) die "move the folder to a path without spaces (now: $DIR)" ;; esac
102 | PY="$(python_bin)" || die "Python 3.8 or newer is needed (python3 not found)"
103 | say "Python: $PY (${PY} --version 2>&1)"
104 |
105 | VERO="$(cd "$DIR" && "$PY" -c 'from vero_status.vero import find_vero; print(find_vero() or "")')"
106 | if [ -n "$VERO" ]; then
107 | _say "Vero CLI: $VERO"
108 | else
109 | _say "WARNING: Vero CLI not found. Install it (npm), run 'vero auth', or set its path later in Settings."
110 | _fi
111 |
112 | if [ "$ACTION" = team ]; then
113 | _PORT="${2:-8767}"
114 | _case "$PORT" in *[!0-9]*) die "port must be a number" ;; esac
115 | _CMD=("$PY" "$DIR/vero_status_server.py" --port "$PORT")
116 | else
117 | _PORT="$(port_of)"
118 | _CMD=("$PY" "$DIR/Vero Status.pyw")
119 | _fi
120 |
121 | uninstall >/dev/null _# one autostart at a time
122 | write_env
123 |
124 | if have_systemd; then
125 | _mkdir -p "$UNIT_DIR"
126 | _EXEC=""
127 | _for part in "${CMD[@]}"; do EXEC+="${part}\n "; done
128 | _cat > "$UNIT" <<EOF
129 | [Unit]
130 | Description=Vero Status ($ACTION) - ASPF-1578
131 | After=network-online.target
132 |
133 | [Service]

```

```

134 | Type=simple
135 | WorkingDirectory=$DIR
136 | EnvironmentFile=$ENV_FILE
137 | ExecStart=$EXEC
138 | Restart=on-failure
139 | RestartSec=30
140 |
141 | [Install]
142 | WantedBy=default.target
143 | EOF
144 | systemctl --user daemon-reload
145 | systemctl --user enable --now vero-status.service
146 | say "Installed the systemd user service 'vero-status'."
147 | if [ "$(logindctl show-user "$USER" -p Linger --value 2>/dev/null || echo no)" != yes ]; then
148 |     if logindctl enable-linger "$USER" 2>/dev/null; then
149 |         say "Enabled lingering: it also runs when you are not logged in."
150 |     else
151 |         say "NOTE: it runs while you are logged in (a VS Code SSH window counts). To keep it running after reboots"
152 |         say "without a login, ask the VM admin once: sudo logindctl enable-linger $USER"
153 |     fi
154 | fi
155 | elif command -v crontab >/dev/null; then
156 |     LINE="@reboot cd \"$DIR\" && set -a && . \"$ENV_FILE\" && set +a && nohup ${CMD[*]}@Q >/dev/null 2>&1 & $TAG"
157 |     { crontab -l 2>/dev/null || true; echo "$LINE"; } | crontab -
158 |     # shellcheck source=/dev/null
159 |     (cd "$DIR" && set -a && . "$ENV_FILE" && set +a && setsid nohup "${CMD[@]}" &) >/dev/null 2>&1 < /dev/null # detach fully: an ssh or pipe reader must not wait
160 |     say "No systemd user session: added a crontab @reboot line and started it now."
161 | else
162 |     (cd "$DIR" && setsid nohup "${CMD[@]}" &) >/dev/null 2>&1 < /dev/null # detach fully: an ssh or pipe reader must not wait
163 |     say "WARNING: neither systemd nor crontab is available: started it now, but it will not start after a reboot."
164 | fi
165 |
166 | if wait_healthy "$PORT"; then
167 |     say ""
168 |     say "Vero Status is running on this VM, port $PORT. Open it on your laptop:"
169 |     say "VS Code (Remote-SSH): PORTS tab > Forward a Port > $PORT, then open http://localhost:$PORT"
170 |     say "or a terminal: ssh -N -L $PORT:127.0.0.1:$PORT $USER@(hostname -f 2>/dev/null || hostname)"
171 |     if [ "$ACTION" = team ]; then
172 |         say "Team link (needs port $PORT open to the VM): http://$(hostname -f 2>/dev/null || hostname):$PORT/"
173 |         say "Admin key: $DIR/data/admin_key.txt (private)"
174 |     fi
175 |     say "Log: $DIR/data/vero-status.log"
176 | else
177 |     die "it did not start; see $DIR/data/vero-status.log"
178 | fi
=== END FILE: scripts/install_vm.sh ===

```

```
=== FILE: scripts/vero (2 lines) ===
  1 | #!/bin/sh
  2 | exec python3 "${dirname "$0"}/fake_vero.py" "$@"
=== END FILE: scripts/vero ===
```

```
=== FILE: scripts/vero.cmd (2 lines) ===  
1 | @echo off  
2 | py -3 "%~dp0fake_vero.py" %*  
=== END FILE: scripts/vero.cmd ===
```

```
=== FILE: tests/__init__.py (1 lines) ===
  1 | """Tests (never call the real Vero)."""
=== END FILE: tests/__init__.py ===
```

```

=== FILE: tests/helpers.py (29 lines) ===
1 | """Shared helpers: run the bundled fake Vero CLI through the real process code."""
2 |
3 | import sys
4 | from pathlib import Path
5 |
6 | ROOT = Path(__file__).resolve().parents[1]
7 | sys.path.insert(0, str(ROOT))
8 |
9 | from vero_status import vero_# noqa: E402
10 |
11 | FAKE = ROOT / "scripts" / "fake_vero.py"
12 |
13 |
14 | def fake_run(args, timeout_s, cwd=None):
15 |     """Like vero.run, but 'vero' is the fake script started with this Python."""
16 |     return vero.run([sys.executable, str(FAKE), *args[1:]], timeout_s, cwd)
17 |
18 |
19 | def quiet_scanner():
20 |     """A session scanner that looks nowhere (tests must not read the real ~/.vero)."""
21 |     from vero_status.sessions import Scanner
22 |
23 |     return Scanner(roots=[], list_processes=lambda: [])
24 |
25 |
26 | def fake_settings(**over):
27 |     from vero_status.settings import Settings
28 |
29 |     return Settings(vero_path=sys.executable, **over)
=== END FILE: tests/helpers.py ===

```

```
=== FILE: tests/test_monitor_server.py (310 lines) ===
```

```

1 | import json
2 | import os
3 | import socket
4 | import tempfile
5 | import threading
6 | import time
7 | import unittest
8 | import urllib.error
9 | import urllib.request
10 | from pathlib import Path
11 | from unittest import mock
12 |
13 | from tests.helpers import fake_run, fake_settings, quiet_scanner
14 | from vero_status.monitor import Monitor, perform_check
15 | from vero_status.server import create
16 |
17 |
18 | def free_port():
19 |     with socket.socket() as s:
20 |         s.bind(("127.0.0.1", 0))
21 |         return s.getsockname()[1]
22 |
23 |
24 | class PerformCheckTest(unittest.TestCase):
25 |     def setUp(self):
26 |         self.tmp = tempfile.TemporaryDirectory()
27 |         self.work = Path(self.tmp.name) / "sandbox"
28 |         self.env = mock.patch.dict(os.environ, {"FAKE_VERO_DELAY": "0.05", "FAKE_VERO_MODE": "ok"})
29 |         self.env.start()
30 |
31 |     def tearDown(self):
32 |         self.env.stop()
33 |         self.tmp.cleanup()
34 |
35 |     def check(self, mode="ok", **settings):
36 |         os.environ["FAKE_VERO_MODE"] = mode
37 |         return perform_check(fake_settings(**settings), self.work, fake_run)
38 |
39 |     def test_available_with_both_models(self):
40 |         r = self.check()
41 |         self.assertTrue(r["available"], r["reason"])
42 |         self.assertEqual(r["asked_model"], "us.anthropic.claude-haiku-4-5-20251001-v1:0")_# pinned cheap model
43 |         self.assertEqual(r["configured"]["model"], "us.anthropic.claude-opus-5")_# Vero's own setting
44 |         self.assertEqual(r["vero_version"], "vero 2.3.3 (fake0000)")
45 |         self.assertEqual(r["reason"], "")
46 |         self.assertIsNotNone(r["seconds"])
47 |
48 |     def test_empty_model_uses_veros_own(self):
49 |         self.assertEqual(self.check(check_model="")["asked_model"], "us.anthropic.claude-opus-5")
50 |
51 |     def test_failures_are_not_available_with_reason(self):
52 |         down = self.check("down")
53 |         self.assertEqual(down["reason"], "Vero reported an error (exit code 1): provider unavailable (connect ETIMEDOUT)")
54 |         self.assertIsNone(down["seconds"])_# no answer, no response time
55 |         r = self.check("auth")
56 |         self.assertFalse(r["available"])
57 |         self.assertNotIn("abc123secret", r["reason"])
58 |         self.assertEqual(self.check("nocompletion")["reason"], "Vero finished without an answer")
59 |
60 |     def test_refusal_is_available_with_note(self):
61 |         r = self.check("refuse")
62 |         self.assertTrue(r["available"])
63 |         self.assertIn("not with OK", r["reason"])
64 |
65 |     def test_vero_missing(self):
66 |         with mock.patch("vero_status.monitor.vero.find_vero", return_value=None):

```

```

67 | r = perform_check(fake_settings(), self.work, lambda *a: self.fail("must not run"))
68 | self.assertFalse(r["available"])
69 | self.assertTrue(r["reason"].startswith("Vero CLI not found"))
70 |
71 | def test_hang_times_out(self):
72 |     os.environ["FAKE_VERO_MODE"] = "hang"
73 |     s = fake_settings(timeout_seconds=30)
74 |     t0 = time.time()
75 |     r = perform_check(s, self.work, lambda a, t, c=None: fake_run(a, min(t, 1.0), c))_# shorten the wait
76 |     self.assertFalse(r["available"])
77 |     self.assertIn("no answer within", r["reason"])
78 |     self.assertLess(time.time() - t0, 6)
79 |
80 | def test_secret_prompt_never_in_record(self):
81 |     self.assertNotIn("FAKE-SECRET-PROMPT", json.dumps(self.check()))
82 |
83 |
84 | class MonitorTest(unittest.TestCase):
85 |     def setUp(self):
86 |         self.tmp = tempfile.TemporaryDirectory()
87 |         self.dir = Path(self.tmp.name)
88 |         self.env = mock.patch.dict(os.environ, {"FAKE_VERO_DELAY": "0.05", "FAKE_VERO_MODE": "ok"})
89 |         self.env.start()
90 |
91 |     def tearDown(self):
92 |         self.env.stop()
93 |         self.tmp.cleanup()
94 |
95 |     def wait_idle(self, m):
96 |         for _ in range(100):
97 |             if not m.checking:
98 |                 return
99 |             time.sleep(0.05)
100 |         self.fail("check did not finish")
101 |
102 |     def make(self):
103 |         m = Monitor(self.dir, fake_run, quiet_scanner())
104 |         m.settings = fake_settings()
105 |         return m
106 |
107 |     def test_check_history_state_and_persistence(self):
108 |         m = self.make()
109 |         self.assertTrue(m.check_now())
110 |         self.assertFalse(m.check_now())_# one at a time
111 |         self.wait_idle(m)
112 |         st = m.state()
113 |         self.assertTrue(st["last"]["available"])
114 |         self.assertEqual(st["availability_24h"], st["checks_24h"], (100.0, 1))
115 |         self.assertEqual(st["configured"]["model"], "us.anthropic.claude-opus-5")
116 |         self.assertIsNotNone(st["next_check"])
117 |         self.assertEqual(len(Monitor(self.dir, fake_run, quiet_scanner()).history), 1)_# saved to disk
118 |         m.scan_sessions()
119 |         self.assertEqual(m.state()["sessions"]["tasks_seen"], 0)
120 |         self.assertEqual(m.own_task_ids(), [st["last"]["task_id"]])_# our probe task is remembered
121 |
122 |     def test_since_tracks_the_current_streak(self):
123 |         m = self.make()
124 |         m.history = [{"time": 10, "available": True}, {"time": 20, "available": False}, {"time": 30, "available": False}]
125 |         self.assertEqual(m.state()["since"], 20)
126 |
127 |     def test_settings_update_and_interval_off(self):
128 |         m = self.make()
129 |         m.update_settings({"interval_minutes": 0})
130 |         self.assertIsNone(m.next_check)
131 |         self.assertEqual(json.loads((self.dir / "settings.json").read_text())["interval_minutes"], 0)
132 |
133 |     def test_scheduler_runs_when_due(self):

```

```

134 |         m = self.make()
135 |         m.start(check_at_start=False)
136 |         m.next_check = time.time() + 0.2
137 |         m._wake.set()
138 |         for _ in range(100):
139 |             if m.history:
140 |                 break
141 |             time.sleep(0.05)
142 |         m.stop()
143 |         self.assertEqual(m.history[-1]["trigger"], "auto")
144 |
145 |
146 | class ServerTest(unittest.TestCase):
147 |     def setUp(self):
148 |         self.tmp = tempfile.TemporaryDirectory()
149 |         self.env = mock.patch.dict(os.environ, {"FAKE_VERO_DELAY": "0.05", "FAKE_VERO_MODE": "ok"})
150 |         self.env.start()
151 |         self.port = free_port()
152 |         self.mon = Monitor(Path(self.tmp.name), fake_run, quiet_scanner())
153 |         self.mon.settings = fake_settings()
154 |         self.quit = threading.Event()
155 |         self.srv = create(self.mon, self.port, self.quit.set)
156 |         threading.Thread(target=self.srv.serve_forever, daemon=True).start()
157 |         self.base = f"http://127.0.0.1:{self.port}"
158 |
159 |     def tearDown(self):
160 |         self.env.stop()
161 |         if not self.quit.is_set():
162 |             self.srv.shutdown()
163 |         self.srv.server_close()
164 |         self.tmp.cleanup()
165 |
166 |     def req(self, path, data=None, **headers):
167 |         body = json.dumps(data).encode() if data is not None else (b"" if headers.pop("post", False) else None)
168 |         r = urllib.request.Request(self.base + path, data=body, headers={"Content-Type": "application/json", **headers})
169 |         try:
170 |             with urllib.request.urlopen(r) as resp:
171 |                 return resp.status, json.loads(resp.read() or b"null")
172 |         except urllib.error.HTTPError as e:
173 |             return e.code, json.loads(e.read() or b"null")
174 |
175 |     def test_page_state_health(self):
176 |         with urllib.request.urlopen(self.base + "/") as r:
177 |             self.assertIn(b"Vero Status", r.read())
178 |             code, st = self.req("/api/state")
179 |             self.assertEqual((code, st["last"]), (200, None))
180 |             self.assertEqual(self.req("/api/health")[1]["app"], "vero-status")
181 |             self.assertEqual(self.req("/nope")[0], 404)
182 |
183 |     def test_check_now(self):
184 |         self.assertEqual(self.req("/api/check", post=True)[0], 202)
185 |         for _ in range(100):
186 |             if self.mon.history and not self.mon.checking:
187 |                 break
188 |             time.sleep(0.05)
189 |         self.assertTrue(self.req("/api/state")[1]["last"]["available"])
190 |
191 |     def test_settings_endpoint(self):
192 |         self.assertEqual(self.req("/api/settings", {"interval_minutes": 30})[0], 200)
193 |         self.assertEqual(self.mon.settings.interval_minutes, 30)
194 |         code, body = self.req("/api/settings", {"interval_minutes": 7})
195 |         self.assertEqual(code, 400)
196 |         self.assertIn("interval", body["error"])
197 |         self.assertEqual(self.req("/api/settings", [1])[0], 400)
198 |
199 |     def test_foreign_host_and_origin_rejected(self):
200 |         self.assertEqual(self.req("/api/state", Host="evil.example")[0], 403)

```

```

201 | self.assertEqual(self.req("/api/state", Host="evil.example:8767")[0], 403)
202 | self.assertEqual(self.req("/api/state", Host="localhost.evil.example")[0], 403)
203 |
204 | def test_forwarded_from_another_port(self):
205 |     """VS Code port forwarding or `ssh -L 9000:127.0.0.1:8767` from a VM: the browser's port differs."""
206 |     self.assertEqual(self.req("/api/state", Host="localhost:9000")[0], 200)
207 |     self.assertEqual(self.req("/api/check", post=True, Host="localhost:9000", Origin="http://localhost:9000")[0], 202)
208 |     self.assertEqual(self.req("/api/check", post=True, Host="localhost:9000", Origin="http://localhost:8767")[0], 403)
209 |     self.assertEqual(self.req("/api/check", post=True, Origin="http://evil.example")[0], 403)
210 |     self.assertEqual(self.req("/api/quit", post=True, Origin="http://evil.example")[0], 403)
211 |
212 | def test_quit(self):
213 |     self.assertEqual(self.req("/api/quit", post=True)[1], {"stopping": True})
214 |     self.assertTrue(self.quit.wait(5))
215 |
216 |
217 | if __name__ == "__main__":
218 |     unittest.main()
219 |
220 |
221 | class ServerModeTest(unittest.TestCase):
222 |     KEY = "test-admin-key-0123456789"
223 |
224 |     def setUp(self):
225 |         self.tmp = tempfile.TemporaryDirectory()
226 |         self.env = mock.patch.dict(os.environ, {"FAKE_VERO_DELAY": "0.05", "FAKE_VERO_MODE": "ok"})
227 |         self.env.start()
228 |         self.port = free_port()
229 |         self.mon = Monitor(Path(self.tmp.name), fake_run, quiet_scanner(), server_mode=True)
230 |         self.mon.settings = fake_settings()
231 |         self.srv = create(self.mon, self.port, server_mode=True, admin_key=self.KEY, host="127.0.0.1")
232 |         threading.Thread(target=self.srv.serve_forever, daemon=True).start()
233 |         self.base = f"http://127.0.0.1:{self.port}"
234 |
235 |     def tearDown(self):
236 |         self.env.stop()
237 |         self.srv.shutdown()
238 |         self.srv.server_close()
239 |         self.tmp.cleanup()
240 |
241 |     def req(self, path, data=None, post=False, **headers):
242 |         body = json.dumps(data).encode() if data is not None else (b"" if post else None)
243 |         r = urllib.request.Request(self.base + path, data=body, headers={"Content-Type": "application/json", **headers})
244 |         try:
245 |             with urllib.request.urlopen(r) as resp:
246 |                 return resp.status, json.loads(resp.read() or b"null")
247 |         except urllib.error.HTTPError as e:
248 |             return e.code, json.loads(e.read() or b"null")
249 |
250 |     def wait_idle(self):
251 |         for _ in range(100):
252 |             if not self.mon.checking:
253 |                 return
254 |             time.sleep(0.05)
255 |
256 |     def test_anyone_can_read_without_login_any_host_name(self):
257 |         code, st = self.req("/api/state", Host=f"vero-status.example.nxp.com:{self.port}")
258 |         self.assertEqual(code, 200)
259 |         self.assertEqual((st["mode"], st["admin"]), ("server", False))
260 |         self.assertEqual((st["settings"]["vero_path"], st["settings"]["vero_found"]), ("", None)) # server paths hidden
261 |         self.assertIsNone(st["sessions"]) # no personal panels on a shared server
262 |         self.assertFalse(st["settings"]["track_sessions"])
263 |
264 |     def test_admin_key_unlocks_settings(self):
265 |         self.assertEqual(self.req("/api/settings", {"interval_minutes": 30})[0], 403)
266 |         self.assertEqual(self.req("/api/settings", {"interval_minutes": 30}, **{"X-Admin-Key": "wrong"})[0], 403)
267 |         self.assertEqual(self.req("/api/settings", {"interval_minutes": 30}, **{"X-Admin-Key": self.KEY})[0], 200)

```

```

268 |         self.assertEqual(self.mon.settings.interval_minutes, 30)
269 |         st = self.req("/api/state", **{"X-Admin-Key": self.KEY})
270 |         self.assertTrue(st["admin"])
271 |         self.assertTrue(st["settings"]["vero_found"])
272 |
273 |     def test_no_quit_on_the_server(self):
274 |         self.assertEqual(self.req("/api/quit", post=True)[0], 404)
275 |         self.assertEqual(self.req("/api/quit", post=True, **{"X-Admin-Key": self.KEY})[0], 404)
276 |         self.assertEqual(self.req("/api/health")[0], 200)
277 |
278 |     def test_check_now_is_rate_limited_for_viewers_not_admin(self):
279 |         self.assertEqual(self.req("/api/check", post=True)[0], 202)
280 |         self.wait_idle()
281 |         code, body = self.req("/api/check", post=True)
282 |         self.assertEqual(code, 429)
283 |         self.assertIn("try again", body["error"])
284 |         self.assertGreater(self.req("/api/state")[1]["check_wait_s"], 0)
285 |         self.assertEqual(self.req("/api/check", post=True, **{"X-Admin-Key": self.KEY})[0], 202)
286 |
287 |     def test_cross_site_post_still_blocked(self):
288 |         self.assertEqual(self.req("/api/check", post=True, Origin="http://evil.example")[0], 403)
289 |
290 |
291 | class AdminKeyTest(unittest.TestCase):
292 |     def test_created_once_and_reused(self):
293 |         from vero_status.app import admin_key
294 |
295 |         with tempfile.TemporaryDirectory() as d:
296 |             k1 = admin_key(Path(d))
297 |             self.assertGreaterEqual(len(k1), 24)
298 |             self.assertEqual(admin_key(Path(d)), k1)
299 |             if os.name != "nt":
300 |                 # a shared Linux VM: other users must not read the admin key
301 |                 self.assertEqual((Path(d) / "admin_key.txt").stat().st_mode & 0o077, 0)
302 |
303 | class HeadlessTest(unittest.TestCase):
304 |     def test_no_browser_on_a_headless_linux_vm(self):
305 |         from vero_status.app import can_open_browser
306 |
307 |         self.assertFalse(can_open_browser("linux", {}))
308 |         self.assertTrue(can_open_browser("linux", {"DISPLAY": ":0"}))
309 |         self.assertTrue(can_open_browser("linux", {"WAYLAND_DISPLAY": "wayland-0"}))
310 |         self.assertTrue(can_open_browser("win32", {}))
311 |
312 |
313 |
314 |
315 |
316 |
317 |
318 |
319 |
320 |
321 |
322 |
323 |
324 |
325 |
326 |
327 |
328 |
329 |
330 |
331 |
332 |
333 |
334 |
335 |
336 |
337 |
338 |
339 |
340 |
341 |
342 |
343 |
344 |
345 |
346 |
347 |
348 |
349 |
350 |
351 |
352 |
353 |
354 |
355 |
356 |
357 |
358 |
359 |
360 |
361 |
362 |
363 |
364 |
365 |
366 |
367 |
368 |
369 |
370 |
371 |
372 |
373 |
374 |
375 |
376 |
377 |
378 |
379 |
380 |
381 |
382 |
383 |
384 |
385 |
386 |
387 |
388 |
389 |
390 |
391 |
392 |
393 |
394 |
395 |
396 |
397 |
398 |
399 |
400 |
401 |
402 |
403 |
404 |
405 |
406 |
407 |
408 |
409 |
410 |
411 |
412 |
413 |
414 |
415 |
416 |
417 |
418 |
419 |
420 |
421 |
422 |
423 |
424 |
425 |
426 |
427 |
428 |
429 |
430 |
431 |
432 |
433 |
434 |
435 |
436 |
437 |
438 |
439 |
440 |
441 |
442 |
443 |
444 |
445 |
446 |
447 |
448 |
449 |
450 |
451 |
452 |
453 |
454 |
455 |
456 |
457 |
458 |
459 |
460 |
461 |
462 |
463 |
464 |
465 |
466 |
467 |
468 |
469 |
470 |
471 |
472 |
473 |
474 |
475 |
476 |
477 |
478 |
479 |
480 |
481 |
482 |
483 |
484 |
485 |
486 |
487 |
488 |
489 |
490 |
491 |
492 |
493 |
494 |
495 |
496 |
497 |
498 |
499 |
500 |
501 |
502 |
503 |
504 |
505 |
506 |
507 |
508 |
509 |
510 |
511 |
512 |
513 |
514 |
515 |
516 |
517 |
518 |
519 |
520 |
521 |
522 |
523 |
524 |
525 |
526 |
527 |
528 |
529 |
530 |
531 |
532 |
533 |
534 |
535 |
536 |
537 |
538 |
539 |
540 |
541 |
542 |
543 |
544 |
545 |
546 |
547 |
548 |
549 |
550 |
551 |
552 |
553 |
554 |
555 |
556 |
557 |
558 |
559 |
560 |
561 |
562 |
563 |
564 |
565 |
566 |
567 |
568 |
569 |
570 |
571 |
572 |
573 |
574 |
575 |
576 |
577 |
578 |
579 |
580 |
581 |
582 |
583 |
584 |
585 |
586 |
587 |
588 |
589 |
590 |
591 |
592 |
593 |
594 |
595 |
596 |
597 |
598 |
599 |
600 |
601 |
602 |
603 |
604 |
605 |
606 |
607 |
608 |
609 |
610 |
611 |
612 |
613 |
614 |
615 |
616 |
617 |
618 |
619 |
620 |
621 |
622 |
623 |
624 |
625 |
626 |
627 |
628 |
629 |
630 |
631 |
632 |
633 |
634 |
635 |
636 |
637 |
638 |
639 |
640 |
641 |
642 |
643 |
644 |
645 |
646 |
647 |
648 |
649 |
650 |
651 |
652 |
653 |
654 |
655 |
656 |
657 |
658 |
659 |
660 |
661 |
662 |
663 |
664 |
665 |
666 |
667 |
668 |
669 |
670 |
671 |
672 |
673 |
674 |
675 |
676 |
677 |
678 |
679 |
680 |
681 |
682 |
683 |
684 |
685 |
686 |
687 |
688 |
689 |
690 |
691 |
692 |
693 |
694 |
695 |
696 |
697 |
698 |
699 |
700 |
701 |
702 |
703 |
704 |
705 |
706 |
707 |
708 |
709 |
710 |
711 |
712 |
713 |
714 |
715 |
716 |
717 |
718 |
719 |
720 |
721 |
722 |
723 |
724 |
725 |
726 |
727 |
728 |
729 |
730 |
731 |
732 |
733 |
734 |
735 |
736 |
737 |
738 |
739 |
740 |
741 |
742 |
743 |
744 |
745 |
746 |
747 |
748 |
749 |
750 |
751 |
752 |
753 |
754 |
755 |
756 |
757 |
758 |
759 |
760 |
761 |
762 |
763 |
764 |
765 |
766 |
767 |
768 |
769 |
770 |
771 |
772 |
773 |
774 |
775 |
776 |
777 |
778 |
779 |
780 |
781 |
782 |
783 |
784 |
785 |
786 |
787 |
788 |
789 |
790 |
791 |
792 |
793 |
794 |
795 |
796 |
797 |
798 |
799 |
800 |
801 |
802 |
803 |
804 |
805 |
806 |
807 |
808 |
809 |
810 |
811 |
812 |
813 |
814 |
815 |
816 |
817 |
818 |
819 |
820 |
821 |
822 |
823 |
824 |
825 |
826 |
827 |
828 |
829 |
830 |
831 |
832 |
833 |
834 |
835 |
836 |
837 |
838 |
839 |
840 |
841 |
842 |
843 |
844 |
845 |
846 |
847 |
848 |
849 |
850 |
851 |
852 |
853 |
854 |
855 |
856 |
857 |
858 |
859 |
860 |
861 |
862 |
863 |
864 |
865 |
866 |
867 |
868 |
869 |
870 |
871 |
872 |
873 |
874 |
875 |
876 |
877 |
878 |
879 |
880 |
881 |
882 |
883 |
884 |
885 |
886 |
887 |
888 |
889 |
890 |
891 |
892 |
893 |
894 |
895 |
896 |
897 |
898 |
899 |
900 |
901 |
902 |
903 |
904 |
905 |
906 |
907 |
908 |
909 |
910 |
911 |
912 |
913 |
914 |
915 |
916 |
917 |
918 |
919 |
920 |
921 |
922 |
923 |
924 |
925 |
926 |
927 |
928 |
929 |
930 |
931 |
932 |
933 |
934 |
935 |
936 |
937 |
938 |
939 |
940 |
941 |
942 |
943 |
944 |
945 |
946 |
947 |
948 |
949 |
950 |
951 |
952 |
953 |
954 |
955 |
956 |
957 |
958 |
959 |
960 |
961 |
962 |
963 |
964 |
965 |
966 |
967 |
968 |
969 |
970 |
971 |
972 |
973 |
974 |
975 |
976 |
977 |
978 |
979 |
980 |
981 |
982 |
983 |
984 |
985 |
986 |
987 |
988 |
989 |
990 |
991 |
992 |
993 |
994 |
995 |
996 |
997 |
998 |
999 |
1000 |

```

```

=== END FILE: tests/test_monitor_server.py ===

```

```
=== FILE: tests/test_sessions.py (185 lines) ===
```

```

1 | import json
2 | import os
3 | import tempfile
4 | import time
5 | import unittest
6 | from pathlib import Path
7 |
8 | import tests.helpers_ as # noqa: F401
9 | from vero_status import sessions
10 | from vero_status.vero import PROMPT
11 |
12 | HAIKU = {"providerId": "bedrock", "modelId": "us.anthropic.claude-haiku-4-5-20251001-v1:0", "mode": "act"}
13 | OPUS = {"providerId": "bedrock", "modelId": "us.anthropic.claude-opus-5", "mode": "act"}
14 |
15 |
16 | PROMPT_TEXT = "SECRET PROMPT TEXT"  # stands for the full prompt Vero logs; must never be kept
17 |
18 |
19 | def req(ts, tokens_in=1200, tokens_out=80):
20 |     text = json.dumps({"request": PROMPT_TEXT, "tokensIn": tokens_in, "tokensOut": tokens_out, "cost": 0.01})
21 |     return {"ts": ts, "type": "say", "say": "api_req_started", "text": text, "modelInfo": OPUS}
22 |
23 |
24 | def session(t0_ms, task_text="refactor the parser"):
25 |     """A user's task: two model requests answered after 4.0 s and 2.5 s, then a completion."""
26 |     return [
27 |         {"ts": t0_ms, "type": "say", "say": "task", "text": task_text, "modelInfo": OPUS},
28 |         req(t0_ms + 100),
29 |         {"ts": t0_ms + 4100, "type": "say", "say": "text", "text": "Let me look", "partial": True, "modelInfo": OPUS},
30 |         {"ts": t0_ms + 9000, "type": "ask", "ask": "tool", "text": "read file"},
31 |         req(t0_ms + 20000),
32 |         {"ts": t0_ms + 20000 + 2500, "type": "say", "say": "completion_result", "text": "done", "modelInfo": OPUS},
33 |     ]
34 |
35 |
36 | class ParseTest(unittest.TestCase):
37 |     def test_latency_per_request_from_veros_timestamps(self):
38 |         s = sessions.parse_ui_messages(session(1_000_000), "t1", "CLI", now=2000)
39 |         self.assertEqual([r.seconds for r in s.requests], [4.0, 2.5])
40 |         self.assertEqual((s.model, s.provider), ("us.anthropic.claude-opus-5", "bedrock"))
41 |         self.assertEqual((s.requests[0].tokens_in, s.requests[0].tokens_out), (1200, 80))
42 |         self.assertAlmostEqual(s.last_activity, 1022.5)
43 |         self.assertIsNone(s.waiting_since)
44 |         self.assertFalse(s.first_prompt_is_probe)
45 |
46 |     def test_prompt_and_answer_text_never_kept(self):
47 |         s = sessions.parse_ui_messages(session(1_000_000), "t1", "CLI", now=2000)
48 |         summary = sessions.summarize([s], False, 2000, [])
49 |         blob = repr(s) + json.dumps(summary)
50 |         for text in ("SECRET PROMPT TEXT", "refactor the parser", "Let me look", "read file"):
51 |             self.assertNotIn(text, blob)
52 |
53 |     def test_request_in_progress_and_failed(self):
54 |         evs = [*session(1_000_000)[:2], req(1_030_000)]
55 |         s = sessions.parse_ui_messages(evs, "t", "CLI", now=1040)
56 |         self.assertEqual(s.waiting_since, 1030)
57 |         failed = [req(1_000_000), {"ts": 1_003_000, "type": "ask", "ask": "api_req_failed", "text": "429"}]
58 |         self.assertTrue(sessions.parse_ui_messages(failed, "t", "CLI", now=2000).requests[0].failed)
59 |
60 |     def test_long_pause_and_bad_events_ignored(self):
61 |         evs = [req(0), {"ts": 31 * 60 * 1000, "say": "text", "junk": {"no_ts": 1}, {"ts": "x"}}]
62 |         s = sessions.parse_ui_messages(evs, "t", "CLI", now=99999)
63 |         self.assertEqual(s.requests, [])
64 |
65 |     def test_probe_task_recognised(self):
66 |         evs = session(1_000_000, task_text=PROMPT)

```

```

67 |         self.assertTrue(sessions.parse_ui_messages(evs, "t", "CLI").first_prompt_is_probe)
68 |
69 |     def test_percentile(self):
70 |         self.assertIsNone(sessions.percentile([], 50))
71 |         self.assertEqual(sessions.percentile([1, 2, 3, 4], 50), 2.5)
72 |         self.assertEqual(sessions.percentile([5], 95), 5)
73 |
74 |
75 | class ProcessDetectionTest(unittest.TestCase):
76 |     IGN = ("C:\\Tools\\vero-status\\data\\sandbox", "vero_status")
77 |
78 |     def test_user_vero_cli(self):
79 |         for cmd in (
80 |             "C:\\Program Files\\nodejs\\node.exe "
81 |             "C:\\Users\\me\\AppData\\Roaming\\npm\\node_modules\\@nxp\\vero\\bin\\vero.js",
82 |             "node /usr/lib/node_modules/vero/dist/cli.js",
83 |             "vero",
84 |             "vero task fix the build",
85 |             "C:\\npm\\vero.cmd --continue",
86 |         ):
87 |             self.assertTrue(sessions.is_interactive_vero(cmd, self.IGN), cmd)
88 |
89 |     def test_not_user_vero(self):
90 |         for cmd in (
91 |             "node server.js",
92 |             "node build.js --verbose",
93 |             "C:\\npm\\vero.cmd task --json -c C:\\Tools\\vero-status\\data\\sandbox Reply",
94 |             "pythonw C:\\Tools\\vero-status\\vero_status\\app.py",
95 |             "vero version",
96 |             "vero config",
97 |             "python3 Vero Status.pyw",
98 |             "C:\\Python312\\pythonw.exe " "C:\\Tools\\vero-status\\Vero Status.pyw",
99 |             "code --extensions-dir C:\\Users\\me\\.vscode\\extensions\\nxp.vero-1.0",
100 |             "node C:\\work\\averone\\index.js",
101 |             "",
102 |         ):
103 |             self.assertFalse(sessions.is_interactive_vero(cmd, self.IGN), cmd)
104 |
105 |
106 | class ScannerTest(unittest.TestCase):
107 |     def setUp(self):
108 |         self.tmp = tempfile.TemporaryDirectory()
109 |         self.root = Path(self.tmp.name)
110 |         self.cli = self.root / "home" / ".vero" / "data" / "tasks"
111 |         self.code = self.root / "Code" / "User" / "globalStorage" / "nxp.vero-assistant" / "tasks"
112 |         self.procs = []
113 |
114 |     def tearDown(self):
115 |         self.tmp.cleanup()
116 |
117 |     def write(self, base, task_id, events, age_s=0):
118 |         p = base / task_id / "ui_messages.json"
119 |         p.parent.mkdir(parents=True, exist_ok=True)
120 |         p.write_text(json.dumps(events))
121 |         t = time.time() - age_s
122 |         os.utime(p, (t, t))
123 |         return p
124 |
125 |     def scanner(self):
126 |         roots = [(str(self.cli), "CLI"), (str(self.root / "Code" / "User" / "globalStorage" / "*vero*" / "tasks"), "VS Code")]
127 |         return sessions.Scanner(roots, ignore_processes=["sandbox"], list_processes=lambda: self.procs)
128 |
129 |     def test_open_session_latency_and_own_tasks_excluded(self):
130 |         now_ms = int(time.time() * 1000)
131 |         self.write(self.cli, "user1", session(now_ms - 60_000))
132 |         self.write(self.cli, "probe1", session(now_ms - 50_000))  # one of our availability checks (by id)
133 |         self.write(self.cli, "probe2", session(now_ms - 40_000, task_text=PROMPT))  # ours (by prompt)

```

```

134 |         self.scanner().state(own_task_ids=["probe1"])
135 |         self.assertEqual(st["tasks_seen"], 1)
136 |         self.assertTrue(st["open"])
137 |         self.assertEqual(st["current"]["model"], "us.anthropic.claude-opus-5")
138 |         self.assertEqual(st["current"]["source"], "CLI")
139 |         self.assertEqual(st["latency"]["hour"], {"n": 2, "p50": 3.2, "p95": 3.9})
140 |         self.assertEqual([r["s"] for r in st["recent"]], [4.0, 2.5])
141 |
142 |     def test_vscode_extension_storage_found(self):
143 |         now_ms = int(time.time() * 1000)
144 |         self.write(self.code, "vs1", session(now_ms - 30_000))
145 |         self.scanner().state()
146 |         self.assertEqual(st["current"]["source"], "VS Code")
147 |
148 |     def test_idle_history_is_not_open_but_open_cli_is(self):
149 |         old_ms = int((time.time() - 3 * 3600) * 1000)
150 |         self.write(self.cli, "old", session(old_ms), age_s=3 * 3600)
151 |         s = self.scanner()
152 |         st = s.state()
153 |         self.assertFalse(st["open"])
154 |         self.assertIsNone(st["current"])
155 |         self.assertEqual(st["latency"]["day"]["n"], 2)
156 |         self.procs = ["node /x/node_modules/vero/bin/vero.js"]
157 |         st = s.state(force=True)
158 |         self.assertTrue(st["open"] and st["cli_process"])
159 |         self.assertIsNone(st["current"]) # idle CLI: the page shows Vero's configured model
160 |
161 |     def test_cache_and_old_files_skipped(self):
162 |         now_ms = int(time.time() * 1000)
163 |         self.write(self.cli, "week_old", session(now_ms - 9 * 86400_000), age_s=9 * 86400)
164 |         s = self.scanner()
165 |         self.assertEqual(s.state()["tasks_seen"], 0)
166 |         self.assertIs(s.state(), s.state()) # cached between scans
167 |         p = self.write(self.cli, "new", session(now_ms))
168 |         self.assertEqual(s.state(force=True)["tasks_seen"], 1)
169 |         p.write_text("{not json}")
170 |         os.utime(p, (time.time() + 5, time.time() + 5))
171 |         self.assertEqual(s.state(force=True)["tasks_seen"], 0) # a broken file is skipped, not fatal
172 |
173 |     def test_missing_folders(self):
174 |         st = self.scanner().state()
175 |         self.assertEqual((st["open"], st["tasks_seen"], st["recent"]), (False, 0, []))
176 |
177 |
178 | class RootsTest(unittest.TestCase):
179 |     def test_vs_code_remote_ssh_on_a_linux_vm_is_searched(self):
180 |         patterns = [p for p, src in sessions.default_roots() if src == "VS Code"]
181 |         self.assertTrue(any(".vscode-server" in p for p in patterns), patterns)
182 |
183 |
184 | if __name__ == "__main__":
185 |     unittest.main()
=== END FILE: tests/test_sessions.py ===

```

```
=== FILE: tests/test_settings.py (46 lines) ===
```

```

1 | import json
2 | import tempfile
3 | import unittest
4 | from pathlib import Path
5 |
6 | import tests.helpers_ as noqa: F401
7 | from vero_status.settings import CHEAP_MODEL, Settings, SettingsError, load, save
8 |
9 |
10 | class SettingsTest(unittest.TestCase):
11 |     def test_defaults(self):
12 |         s = Settings()
13 |         s.validate()
14 |         self.assertEqual((s.check_model, s.interval_minutes, s.port), (CHEAP_MODEL, 15, 8767))
15 |
16 |     def test_merge_validates(self):
17 |         self.assertEqual(Settings().merged({"interval_minutes": 0, "check_model": ""}).interval_minutes, 0)
18 |         for bad in (
19 |             {"interval_minutes": 7},
20 |             {"interval_minutes": "15"},
21 |             {"timeout_seconds": 5},
22 |             {"port": 80},
23 |             {"check_model": "two words"},
24 |             {"evil": 1},
25 |             {"vero_path": 3},
26 |             {"track_sessions": "yes"},
27 |         ):
28 |             with self.assertRaises(SettingsError, msg=bad):
29 |                 Settings().merged(bad)
30 |
31 |     def test_load_save_and_broken_file(self):
32 |         with tempfile.TemporaryDirectory() as d:
33 |             p = Path(d) / "settings.json"
34 |             self.assertEqual(load(p), Settings())
35 |             save(p, Settings().merged({"interval_minutes": 30}))
36 |             self.assertEqual(load(p).interval_minutes, 30)
37 |             p.write_bytes(b"\xef\xbb\xbf" + json.dumps({"interval_minutes": 60, "old_key": 1}).encode())
38 |             self.assertEqual(load(p).interval_minutes, 60) # BOM and unknown keys tolerated
39 |             p.write_text("{broken}")
40 |             self.assertEqual(load(p), Settings())
41 |             p.write_text('{"interval_minutes": 7}')
42 |             self.assertEqual(load(p), Settings())
43 |
44 |
45 | if __name__ == "__main__":
46 |     unittest.main()

```

```
=== END FILE: tests/test_settings.py ===
```

=== FILE: tests/test_vero.py (124 lines) ===

```

1 | import os
2 | import sys
3 | import tempfile
4 | import time
5 | import unittest
6 | from pathlib import Path
7 | from unittest import mock
8 |
9 | import tests.helpers_ as noqa: F401
10 | from vero_status import vero
11 |
12 | GUIDE_STREAM = [
13 |     {'type': "task_started", "taskId": "1790868292727"},
14 |     {'type': "say", "say": "task", "text": "Reply with exactly: OK", "modelInfo": {"providerId": "bedrock",
15 |     'modelId': "us.anthropic.claude-haiku-4-5-20251001-v1:0", "mode": "act"}},
16 |     {'type': "say", "say": "api_req_started", "text": "\\\"request\\\": \\\"secret prompt\\\""}},
17 |     {'type': "say", "say": "text", "text": "0", "partial": true},
18 |     {'type': "say", "say": "error", "text": "[ERROR] You did not use a tool in your previous response!"},
19 |     {'type': "say", "say": "completion_result", "text": "OK"}
20 | ]
21 |
22 |
23 | class ParseTaskStreamTest(unittest.TestCase):
24 |     def test_stream_from_integration_guide(self):
25 |         s = vero.parse_task_stream(GUIDE_STREAM)
26 |         self.assertEqual((s.answer, s.provider, s.model), ("OK", "bedrock", "us.anthropic.claude-haiku-4-5-20251001-v1:0"))
27 |         self.assertEqual(s.events, 6)
28 |
29 |     def test_only_completion_result_counts_andpartials_ignored(self):
30 |         s = vero.parse_task_stream(['say': "text", "text": "OK"}, {'say': "completion_result", "text": "0", "partial": true}])
31 |         self.assertIsNone(s.answer)
32 |
33 |     def test_garbage_ignored(self):
34 |         s = vero.parse_task_stream(["", "banner", "{bad", "[1]", '{"type": "error", "message": "Bearer abc.def failed"}'])
35 |         self.assertEqual((s.events, s.error), (1, "[hidden] failed"))
36 |
37 |
38 | class ParseConfigTest(unittest.TestCase):
39 |     def test_key_value_lines_as_in_the_guide(self):
40 |         text = (
41 |             "actModeApiProvider : bedrock\n"
42 |             "actModeApiModelId: us.anthropic.claude-opus-5\n"
43 |             "awsRegion: us-west-2\n"
44 |             "other : x"
45 |         )
46 |         self.assertEqual(
47 |             vero.parse_config(text), {"provider": "bedrock", "model": "us.anthropic.claude-opus-5", "region": "us-west-2"}
48 |         )
49 |
50 |     def test_json_and_quoted_variants(self):
51 |         self.assertEqual(vero.parse_config({'actModeApiModelId': "m1", "awsRegion": "eu"}), {"model": "m1", "region": "eu"})
52 |         self.assertEqual(vero.parse_config(' "actModeApiModelId": "m2", ', {"model": "m2"})
53 |         self.assertEqual(vero.parse_config("nothing useful"), {})
54 |
55 |
56 | class SmallParsersTest(unittest.TestCase):
57 |     def test_version_and_clean(self):
58 |         self.assertEqual(vero.parse_version("vero 2.3.3 (35b52374)\nmore"), "vero 2.3.3 (35b52374)")
59 |         self.assertEqual(vero.parse_version(""), "")
60 |         self.assertNotIn("abc123", vero.clean("error token=abc123 password=x"))
61 |         self.assertEqual(len(vero.clean("x" * 999)), vero.DETAIL_MAX)
62 |
63 |     def test_task_args(self):
64 |         a = vero.task_args("vero", "", 120, Path("w"))
65 |         self.assertEqual(a, ["vero", "task", "--json", "-t", "120", "-c", "w", vero.PROMPT])
66 |         self.assertIn("-m", vero.task_args("vero", "m1", 120, Path("w")))

```

```

67 |         self.assertNotIn("--yolo", a)
68 |
69 |
70 |     class RunAndFindTest(unittest.TestCase):
71 |     def test_run_collects_output(self):
72 |         r = vero.run([sys.executable, "-c", "print('hi'); import sys; sys.exit(3)"], 10)
73 |         self.assertEqual((r.started, r.code, r.out.strip(), r.timed_out), (True, 3, "hi", False))
74 |
75 |     def test_run_missing_program(self):
76 |         self.assertFalse(vero.run(["no-such-program-xyz"], 5).started)
77 |
78 |     @unittest.skipIf(os.name == "nt", "POSIX process group")
79 |     def test_timeout_kills_the_whole_tree_fast(self):
80 |         t0 = time.time()
81 |         r = vero.run(["sh", "-c", "sleep 30 & sleep 30; echo late"], 0.5)
82 |         self.assertTrue(r.timed_out)
83 |         self.assertLess(time.time() - t0, 3)
84 |
85 |     def test_find_vero(self):
86 |         self.assertEqual(vero.find_vero(sys.executable), sys.executable)
87 |         with tempfile.TemporaryDirectory() as d:
88 |             no_path = mock.patch("vero_status.vero.shutil.which", return_value=None)
89 |             home = mock.patch("vero_status.vero.Path.home", return_value=Path(d))
90 |             with no_path, home, mock.patch.dict(os.environ, {"APPDATA": ""}), mock.patch.object(
91 |                 Path, "is_file", return_value=False
92 |             ):
93 |                 self.assertIsNone(vero.find_vero("/nope/vero-xyz"))
94 |                 npm = Path(d) / "npm"
95 |                 npm.mkdir()
96 |                 (npm / "vero.cmd").write_text("@echo off")
97 |                 with no_path, home, mock.patch.dict(os.environ, {"APPDATA": d}):
98 |                     self.assertEqual(vero.find_vero(""), str(npm / "vero.cmd")) ## npm default location on Windows
99 |
100 |     def test_find_vero_linux_vm_nvm_newest_node(self):
101 |         """On a VM a service has a minimal PATH: an nvm install must still be found (newest node first)."""
102 |         with tempfile.TemporaryDirectory() as d:
103 |             for v in ("v9.11.2", "v20.11.0", "v18.19.1"):
104 |                 b = Path(d) / ".nvm" / "versions" / "node" / v / "bin"
105 |                 b.mkdir(parents=True)
106 |                 (b / "vero").write_text("#!/usr/bin/env node")
107 |                 with mock.patch("vero_status.vero.shutil.which", return_value=None), mock.patch(
108 |                     "vero_status.vero.Path.home", return_value=Path(d)
109 |                 ), mock.patch.dict(os.environ, {"APPDATA": ""}):
110 |                     self.assertIn("v20.11.0", vero.find_vero("") or "")
111 |
112 |     @unittest.skipIf(os.name == "nt", "POSIX shell script")
113 |     def test_run_puts_the_program_folder_first_on_path(self):
114 |         """So `#!/usr/bin/env node` finds the node that nvm installed next to vero."""
115 |         with tempfile.TemporaryDirectory() as d:
116 |             script = Path(d) / "show-path"
117 |             script.write_text('#!/bin/sh\necho "$PATH\n')
118 |             script.chmod(0o755)
119 |             r = vero.run([str(script)], 10)
120 |             self.assertTrue(r.out.startswith(d + os.pathsep), r.out)
121 |
122 |
123 | if __name__ == "__main__":
124 |     unittest.main()
=== END FILE: tests/test_vero.py ===

```

```
=== FILE: vero_status/__init__.py (4 lines) ===
 1 | """Vero Status: a small dashboard that shows whether Vero is available and which model it uses."""
 2 |
 3 | __version__ = "4.3.0"
 4 | APP_ID = "vero-status"
=== END FILE: vero_status/__init__.py ===
```

```
=== FILE: vero_status/app.py (184 lines) ===
```

```

1 | """Program start: one instance, dashboard in the browser, no console window."""
2 |
3 | from __future__ import annotations
4 |
5 | import argparse
6 | import contextlib
7 | import json
8 | import logging
9 | import logging.handlers
10 | import os
11 | import secrets
12 | import signal
13 | import socket
14 | import sys
15 | import threading
16 | import urllib.request
17 | import webbrowser
18 | from pathlib import Path
19 | from typing import Mapping
20 |
21 | from . import APP_ID
22 | from .monitor import Monitor
23 | from .server import create
24 |
25 | ROOT = Path(__file__).resolve().parents[1]
26 | DATA = ROOT / "data"
27 | log = logging.getLogger("vero_status")
28 |
29 |
30 | def message(text: str) -> None:
31 |     """Tell the user something without a console (pythonw): a Windows message box."""
32 |     if sys.platform == "win32":
33 |         import ctypes
34 |
35 |         ctypes.windll.user32.MessageBoxW(None, text, "Vero Status", 0x40)
36 |     else:
37 |         print(text)
38 |
39 |
40 | def can_open_browser(platform: str = sys.platform, env: Mapping[str, str] | None = None) -> bool:
41 |     """False on a headless Linux VM (no desktop): there the URL is printed and logged instead, and you
42 |     open it through VS Code port forwarding or an SSH tunnel."""
43 |     env = os.environ if env is None else env
44 |     if platform in ("win32", "darwin"):
45 |         return True
46 |     return bool(env.get("DISPLAY") or env.get("WAYLAND_DISPLAY"))
47 |
48 |
49 | def private(path: Path) -> None:
50 |     """Owner-only access on Linux/macOS: a shared VM has other users (no effect on Windows)."""
51 |     if os.name != "nt":
52 |         try:
53 |             path.chmod(0o700 if path.is_dir() else 0o600)
54 |         except OSError:
55 |             log.warning("cannot restrict access to %s", path)
56 |
57 |
58 | def stop_on_sigterm() -> None:
59 |     """systemd and `kill` send SIGTERM: stop like Ctrl+C so the shutdown is logged and clean."""
60 |
61 |     def handler(signum: int, frame: object) -> None:
62 |         raise KeyboardInterrupt
63 |
64 |     with contextlib.suppress(ValueError): # only possible in the main thread
65 |         signal.signal(signal.SIGTERM, handler)
66 |

```

```

67 |
68 | def already_running(port: int) -> bool:
69 |     try:
70 |         with urllib.request.urlopen(f"http://127.0.0.1:{port}/api/health", timeout=2) as r:
71 |             return bool(json.load(r).get("app") == APP_ID)
72 |     except Exception:
73 |         return False
74 |
75 |
76 | def setup_logging() -> None:
77 |     DATA.mkdir(parents=True, exist_ok=True)
78 |     private(DATA)# history, settings, log and the admin key stay readable by you only
79 |     handler = logging.handlers.RotatingFileHandler(DATA / "vero-status.log", maxBytes=500_000, backupCount=2, encoding="utf-8")
80 |     logging.basicConfig(level=logging.INFO, handlers=[handler], format="%(asctime)s %(levelname)s %(message)s", force=True)
81 |
82 |
83 | def main() -> int:
84 |     setup_logging()
85 |     monitor = Monitor(DATA)
86 |     port = monitor.settings.port
87 |     url = f"http://127.0.0.1:{port}/"
88 |     if already_running(port):
89 |         if can_open_browser():
90 |             webbrowser.open(url)# second double-click: just show the dashboard
91 |         else:
92 |             print(f"Vero Status is already running: {url}")
93 |             return 0
94 |     try:
95 |         srv = create(monitor, port)
96 |     except OSError as e:
97 |         log.error("cannot use port %s: %s", port, e)
98 |         message(
99 |             f"Vero Status cannot start: port {port} is used by another program.\n\n"
100 |             f'Change "port" in {DATA / "settings.json"} and start it again.'
101 |         )
102 |         return 1
103 |     monitor.start(check_at_start=True)
104 |     log.info("dashboard %s", url)
105 |     if can_open_browser():
106 |         threading.Timer(0.3, webbrowser.open, args=(url,)).start()
107 |     else:# headless VM: never start a text-mode browser in the background
108 |         print(f"Vero Status runs at {url} (forward port {port} to open it on your laptop)")
109 |     stop_on_sigterm()
110 |     try:
111 |         srv.serve_forever()
112 |     except KeyboardInterrupt:
113 |         pass
114 |     finally:
115 |         monitor.stop()
116 |         srv.server_close()
117 |         log.info("stopped")
118 |         return 0
119 |
120 |
121 | def admin_key(data: Path) -> str:
122 |     """The server's admin key: created once, kept in data/admin_key.txt (keep this file private)."""
123 |     path = data / "admin_key.txt"
124 |     try:
125 |         key = path.read_text(encoding="utf-8").strip()
126 |         if len(key) >= 16:
127 |             return key
128 |     except OSError:
129 |         pass
130 |     data.mkdir(parents=True, exist_ok=True)
131 |     key = secrets.token_urlsafe(24)
132 |     path.write_text(key + "\n", encoding="utf-8")
133 |     private(path)

```

```

134 |     return key
135 |
136 |
137 | def serve_team(argv: list[str] | None = None) -> int:
138 |     """Server mode: one shared status page for everyone on the network, no login."""
139 |     ap = argparse.ArgumentParser(description="Vero Status team server")
140 |     ap.add_argument("--port", type=int, help="default: port in data/settings.json (8767)")
141 |     ap.add_argument("--host", default="0.0.0.0", help="address to listen on (default: all)")
142 |     a = ap.parse_args(argv)
143 |     setup_logging()
144 |     # a terminal shows the links; under systemd or cron (no terminal) they go to data/vero-status.log only,
145 |     # so the admin key does not end up in the system journal, which admins can read
146 |     console = logging.StreamHandler(sys.stdout) if sys.stdout and sys.stdout.isatty() else None
147 |     if console:
148 |         logging.getLogger().addHandler(console)
149 |     monitor = Monitor(DATA, server_mode=True)
150 |     port = a.port or monitor.settings.port
151 |     key = admin_key(DATA)
152 |     try:
153 |         srv = create(monitor, port, server_mode=True, admin_key=key, host=a.host)
154 |     except OSError as e:
155 |         log.error("cannot listen on %s:%s: %s", a.host, port, e)
156 |         return 1
157 |     fqdn = socket.getfqdn() or socket.gethostname()
158 |     loopback = a.host in ("127.0.0.1", "localhost", "::1")
159 |     name = "127.0.0.1" if loopback else fqdn
160 |     log.info("Vero Status team server %s on %s port %s", fqdn, a.host, port)
161 |     if loopback: # e.g. a cloud VM where only SSH is open
162 |         log.info("Only this machine can connect. From your laptop: ssh -N -L %s:127.0.0.1:%s <user>@%s", port, port, fqdn)
163 |         log.info("Share with the team: http://%s:%s/", name, port)
164 |         log.info("Admin link (keep private, it allows changing settings): http://%s:%s/?admin=%s", name, port, key)
165 |         monitor.start(check_at_start=True)
166 |         stop_on_sigterm()
167 |         try:
168 |             srv.serve_forever()
169 |         except KeyboardInterrupt:
170 |             pass
171 |         finally:
172 |             monitor.stop()
173 |             srv.server_close()
174 |             log.info("stopped")
175 |     return 0
176 |
177 |
178 | def run() -> None:
179 |     try:
180 |         sys.exit(main())
181 |     except Exception as e: # last resort: never die silently
182 |         logging.getLogger("vero_status").exception("crash")
183 |         message(f"Vero Status stopped because of an error:\n{e}\n\nDetails: {DATA / 'vero-status.log'}")
184 |         sys.exit(1)
=== END FILE: vero_status/app.py ===

```

```
=== FILE: vero_status/monitor.py (230 lines) ===
```

```

1 | """The monitor: runs a check on a timer or on demand, keeps the latest result and a short history."""
2 |
3 | from __future__ import annotations
4 |
5 | import json
6 | import logging
7 | import socket
8 | import sys
9 | import threading
10 | import time
11 | from pathlib import Path
12 | from typing import Any, Callable, Dict, List, Optional
13 |
14 | from . import sessions, vero
15 | from .settings import Settings, load, save
16 |
17 | log = logging.getLogger("vero_status")
18 | HISTORY_MAX = 500
19 | Runner = Callable[[List[str], float, Optional[str]], vero.Proc]
20 |
21 |
22 | def perform_check(settings: Settings, workdir: Path, run: Runner = vero.run, now: Optional[float] = None) -> Dict[str, Any]:
23 |     """One availability check. Returns a JSON-ready record; never raises for Vero problems."""
24 |     rec: Dict[str, Any] = {
25 |         "time": time.time() if now is None else now,
26 |         "available": False,
27 |         "reason": "",
28 |         "seconds": None,
29 |         "asked_model": None,
30 |         "provider": None,
31 |         "vero_version": None,
32 |         "configured": {},
33 |     }
34 |     exe = vero.find_vero(settings.vero_path)
35 |     if not exe:
36 |         rec["reason"] = "Vero CLI not found on this computer (set its path in Settings)"
37 |         return rec
38 |     ver = run([exe, "version"], 30, None)
39 |     if not ver.started or ver.timed_out or ver.code != 0:
40 |         rec["reason"] = "Vero CLI does not start: " + vero.clean(ver.err or ver.out or "no output")
41 |         return rec
42 |     rec["vero_version"] = vero.parse_version(ver.out)
43 |     cfg = run([exe, "config"], 30, None)
44 |     if cfg.started and cfg.code == 0:
45 |         rec["configured"] = vero.parse_config(cfg.out)
46 |         workdir.mkdir(parents=True, exist_ok=True) # empty folder: the agent has nothing to read or change
47 |         limit = settings.timeout_seconds
48 |         task = run(vero.task_args(exe, settings.check_model, limit, workdir), limit + 30, str(workdir))
49 |         stream = vero.parse_task_stream(task.out.splitlines())
50 |         rec["asked_model"], rec["provider"] = stream.model, stream.provider # model Vero used for the check
51 |         rec["task_id"] = stream.task_id # lets the session tracker ignore our own test tasks
52 |         if task.timed_out:
53 |             rec["reason"] = f"no answer within {limit + 30} s"
54 |         elif stream.answer is not None:
55 |             rec["available"] = True
56 |             rec["seconds"] = round(task.seconds, 1) # only a real answer has a response time
57 |             if "ok" not in stream.answer.lower():
58 |                 rec["reason"] = "answered, but not with OK (a guardrail or the model replied differently)"
59 |             elif stream.error:
60 |                 rec["reason"] = stream.error
61 |             elif task.code:
62 |                 msg = vero.clean(task.err or task.out or "no output")
63 |                 msg = msg[6:].lstrip() if msg.lower().startswith("error:") else msg
64 |                 rec["reason"] = f"Vero reported an error (exit code {task.code}): {msg}"
65 |             elif stream.events == 0:
66 |                 rec["reason"] = "Vero returned no JSON events (is this Vero CLI 2.3 or newer?)"

```

```

67 |     else:
68 |         rec["reason"] = "Vero finished without an answer"
69 |         return rec
70 |
71 |
72 | class Monitor:
73 |     def __init__(
74 |         self, data_dir: Path, run: Runner = vero.run, scanner: Optional[sessions.Scanner] = None, server_mode: bool = False
75 |     ):
76 |         self.data_dir = data_dir
77 |         self.server_mode = server_mode # shared team server: no personal session panels
78 |         self.last_manual: float = 0.0
79 |         self.run = run
80 |         self.settings_path = data_dir / "settings.json"
81 |         self.history_path = data_dir / "history.json"
82 |         self.settings = load(self.settings_path)
83 |         self.history: List[Dict[str, Any]] = self._load_history()
84 |         self.checking = False
85 |         self.next_check: Optional[float] = None
86 |         self._lock = threading.Lock()
87 |         self._wake = threading.Event()
88 |         self._stop = threading.Event()
89 |         self.scanner = scanner or sessions.Scanner(ignore_processes=[str(data_dir / "sandbox"), "vero_status", "vero-status"])
90 |
91 |     # history -----
92 |     def _load_history(self) -> List[Dict[str, Any]]:
93 |         try:
94 |             data = json.loads(self.history_path.read_text(encoding="utf-8"))
95 |             return [r for r in data if isinstance(r, dict) and "time" in r][-HISTORY_MAX:]
96 |         except (OSError, ValueError):
97 |             return []
98 |
99 |     def _save_history(self) -> None:
100 |         self.data_dir.mkdir(parents=True, exist_ok=True)
101 |         tmp = self.history_path.with_suffix(".tmp")
102 |         tmp.write_text(json.dumps(self.history[-HISTORY_MAX:]), encoding="utf-8")
103 |         tmp.replace(self.history_path)
104 |
105 |     # checks -----
106 |     def check_now(self, trigger: str = "manual") -> bool:
107 |         """Start a check in the background. False if one is already running."""
108 |         with self._lock:
109 |             if self.checking:
110 |                 return False
111 |             self.checking = True
112 |             threading.Thread(target=self._check, args=(trigger,), daemon=True).start()
113 |             return True
114 |
115 |     def _check(self, trigger: str) -> None:
116 |         try:
117 |             rec = perform_check(self.settings, self.data_dir / "sandbox", self.run)
118 |         except Exception as e: # a bug must never kill the program
119 |             log.exception("check failed")
120 |             rec = {
121 |                 "time": time.time(),
122 |                 "available": False,
123 |                 "reason": f"monitor error: {type(e).__name__}",
124 |                 "seconds": None,
125 |                 "asked_model": None,
126 |                 "provider": None,
127 |                 "vero_version": None,
128 |                 "configured": {},
129 |             }
130 |             rec["trigger"] = trigger
131 |             with self._lock:
132 |                 self.history.append(rec)
133 |             self.history = self.history[-HISTORY_MAX:]

```

```

134 |         self._save_history()
135 |         self.checking = False
136 |         log.info("check: %s %s", "AVAILABLE" if rec["available"] else "NOT AVAILABLE", rec["reason"])
137 |         self._plan_next()
138 |
139 |     def _plan_next(self) -> None:
140 |         m = self.settings.interval_minutes
141 |         self.next_check = time.time() + m * 60 if m else None
142 |         self._wake.set()
143 |
144 |     def start(self, check_at_start: bool = True) -> None:
145 |         def loop() -> None:
146 |             while not self._stop.is_set():
147 |                 self._wake.clear()
148 |                 due = self.next_check
149 |                 wait = None if due is None else max(0.0, due - time.time())
150 |                 if self._wake.wait(wait) or self._stop.is_set():
151 |                     continue # plan changed: re-evaluate
152 |                 if due is not None and time.time() >= due:
153 |                     self.next_check = None
154 |                     if not self.check_now("auto"):
155 |                         self._plan_next()
156 |
157 |         threading.Thread(target=loop, daemon=True).start()
158 |         threading.Thread(target=self._scan_loop, daemon=True).start()
159 |         if check_at_start:
160 |             self.check_now("start")
161 |         else:
162 |             self._plan_next()
163 |
164 |     def own_task_ids(self) -> List[str]:
165 |         with self._lock:
166 |             return [str(r["task_id"]) for r in self.history if r.get("task_id")]
167 |
168 |     def scan_sessions(self) -> Dict[str, Any]:
169 |         """Your own Vero sessions; never raises (a bad file must not break the dashboard)."""
170 |         try:
171 |             return self.scanner.state(self.own_task_ids(), force=True)
172 |         except Exception:
173 |             log.exception("session scan failed")
174 |             return {}
175 |
176 |     def _scan_loop(self) -> None:
177 |         while not self._stop.is_set():
178 |             if self.settings.track_sessions and not self.server_mode:
179 |                 self.scan_sessions()
180 |             self._stop.wait(sessions.SCAN_EVERY_S)
181 |
182 |     def stop(self) -> None:
183 |         self._stop.set()
184 |         self._wake.set()
185 |
186 |     # settings -----
187 |     def update_settings(self, changes: Dict[str, Any]) -> Settings:
188 |         new = self.settings.merged(changes)
189 |         save(self.settings_path, new)
190 |         self.settings = new
191 |         if not self.checking:
192 |             self._plan_next()
193 |         return new
194 |
195 |     # view -----
196 |     def state(self, admin: bool = True) -> Dict[str, Any]:
197 |         with self._lock:
198 |             hist = list(self.history)
199 |             day = [r for r in hist if r["time"] >= time.time() - 86400]
200 |             last = hist[-1] if hist else None

```

```

201 |         last_ok = next((r for r in reversed(hist) if r.get("available")), None)
202 |         since = None
203 |         if last:
204 |             since = last["time"]
205 |             for r in reversed(hist):
206 |                 if bool(r.get("available")) != bool(last.get("available")):
207 |                     break
208 |             since = r["time"]
209 |         return {
210 |             "checking": self.checking,
211 |             "next_check": self.next_check,
212 |             "last": last,
213 |             "since": since,
214 |             "configured": (last or {}).get("configured") or (last_ok or {}).get("configured") or {},
215 |             "availability_24h": round(sum(1 for r in day if r.get("available")) / len(day) * 100, 1) if day else None,
216 |             "checks_24h": len(day),
217 |             "history": hist[-48:],
218 |             "mode": "server" if self.server_mode else "local",
219 |             "host": socket.gethostname(),UU# where the checks run, e.g. the VM's name
220 |             "platform": sys.platform,
221 |             "settings": {
222 |                 "vero_path": self.settings.vero_path if admin else "",
223 |                 "vero_found": vero.find_vero(self.settings.vero_path) if admin else None,
224 |                 "check_model": self.settings.check_model,
225 |                 "interval_minutes": self.settings.interval_minutes,
226 |                 "timeout_seconds": self.settings.timeout_seconds,
227 |                 "track_sessions": self.settings.track_sessions and not self.server_mode,
228 |             },
229 |             "sessions": self.scanner.result if self.settings.track_sessions and not self.server_mode else None,
230 |         }
=== END FILE: vero_status/monitor.py ===

```

```
=== FILE: vero_status/server.py (140 lines) ===
```

```

1 | """Web server for the dashboard.
2 |
3 | Local mode (default): binds to 127.0.0.1; Host and Origin are checked so other web pages cannot use it.
4 | Server mode (team server): binds to all interfaces, no login. Everyone can read the status; "Check now"
5 | is limited to once per COOLDOWN_S for viewers. Settings and stopping need the admin key, sent in the
6 | X-Admin-Key header (the page takes it once from an admin link). Quit does not exist in server mode.
7 | """
8 |
9 | from __future__ import annotations
10 |
11 | import hmac
12 | import json
13 | import logging
14 | import re
15 | import threading
16 | import time
17 | from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
18 | from pathlib import Path
19 | from typing import Any, Callable, Dict, Optional, Type
20 | from urllib.parse import urlparse
21 |
22 | from . import APP_ID, __version__
23 | from .monitor import Monitor
24 | from .settings import SettingsError
25 |
26 | log = logging.getLogger("vero_status")
27 | INDEX = Path(__file__).parent / "web" / "index.html"
28 | MAX_BODY = 10_000
29 | COOLDOWN_S = 5 * 60_000# server mode: viewers may start a check at most this often
30 |
31 |
32 | def make_handler(
33 |     monitor: Monitor, port: int, on_quit: Callable[[], None], server_mode: bool = False, admin_key: Optional[str] = None
34 | ) -> Type[BaseHTTPRequestHandler]:
35 |     local_names = {"127.0.0.1", "localhost", "[::1]}"_# any port: VS Code or `ssh -L` may forward from another one
36 |
37 |     class Handler(BaseHTTPRequestHandler):
38 |         server_version = f"{APP_ID}/{__version__}"
39 |
40 |         def log_message(self, format: str, *args: Any) -> None:
41 |             log.debug("%s " + format, self.address_string(), *args)
42 |
43 |         def _send(self, code: int, body: bytes, ctype: str) -> None:
44 |             self.send_response(code)
45 |             self.send_header("Content-Type", ctype)
46 |             self.send_header("Content-Length", str(len(body)))
47 |             self.send_header("Cache-Control", "no-store")
48 |             self.send_header("X-Content-Type-Options", "nosniff")
49 |             self.send_header("Referrer-Policy", "no-referrer")_# the admin link never leaks via Referer
50 |             self.end_headers()
51 |             self.wfile.write(body)
52 |
53 |         def _json(self, obj: Any, code: int = 200) -> None:
54 |             self._send(code, json.dumps(obj).encode(), "application/json")
55 |
56 |         def _is_admin(self) -> bool:
57 |             if not server_mode:
58 |                 return True_# local mode: the only user is the person at this computer
59 |             given = self.headers.get("X-Admin-Key") or ""
60 |             return bool(admin_key) and hmac.compare_digest(given.encode(), str(admin_key).encode())
61 |
62 |         def _allowed(self, post: bool) -> bool:
63 |             host = (self.headers.get("Host") or "").lower()
64 |             if not server_mode and re.sub(r":\d+$", "", host) not in local_names:
65 |                 return False_# blocks DNS rebinding: only loopback names, whatever the forwarded port
66 |             origin = self.headers.get("Origin")

```

```

67 |         return not post or origin is None or urlparse(origin).netloc.lower() == host
68 |
69 |     def do_GET(self) -> None:
70 |         if not self._allowed(post=False):
71 |             return self._json({"error": "forbidden"}, 403)
72 |         path = urlparse(self.path).path
73 |         if path in ("/", "/index.html"):
74 |             self._send(200, INDEX.read_bytes(), "text/html; charset=utf-8")
75 |         elif path == "/api/state":
76 |             admin = self._is_admin()
77 |             cooldown = max(0.0, monitor.last_manual + COOLDOWN_S - time.time()) if server_mode and not admin else 0.0
78 |             self._json({"version": __version__, "admin": admin, "check_wait_s": round(cooldown, **monitor.state(admin))})
79 |         elif path == "/api/health":
80 |             self._json({"app": APP_ID, "version": __version__})
81 |         else:
82 |             self._json({"error": "not found"}, 404)
83 |
84 |     def do_POST(self) -> None:
85 |         if not self._allowed(post=True):
86 |             return self._json({"error": "forbidden"}, 403)
87 |         path = urlparse(self.path).path
88 |         admin = self._is_admin()
89 |         if path == "/api/check":
90 |             wait = monitor.last_manual + COOLDOWN_S - time.time()
91 |             if server_mode and not admin and wait > 0:
92 |                 return self._json({"error": f"checked recently; try again in {int(wait // 60) + 1} min"}, 429)
93 |             started = monitor.check_now("manual")
94 |             if started:
95 |                 monitor.last_manual = time.time()
96 |             return self._json({"started": started}, 202 if started else 409)
97 |         if path == "/api/settings":
98 |             if not admin:
99 |                 return self._json({"error": "settings can only be changed with the admin link"}, 403)
100 |             try:
101 |                 length = int(self.headers.get("Content-Length") or 0)
102 |                 if not 0 < length <= MAX_BODY:
103 |                     raise SettingsError("empty or too large request")
104 |                 changes = json.loads(self.rfile.read(length))
105 |                 if not isinstance(changes, dict):
106 |                     raise SettingsError("expected a JSON object")
107 |                 monitor.update_settings(changes)
108 |             except (SettingsError, ValueError, TypeError) as e:
109 |                 return self._json({"error": str(e)}, 400)
110 |             return self._json({"saved": True})
111 |             if path == "/api/quit" and not server_mode:
112 |                 self._json({"stopping": True})
113 |                 threading.Thread(target=on_quit, daemon=True).start()
114 |             return None
115 |             return self._json({"error": "not found"}, 404)
116 |
117 |     return Handler
118 |
119 |
120 | def create(
121 |     monitor: Monitor,
122 |     port: int,
123 |     on_quit: Optional[Callable[[], None]] = None,
124 |     server_mode: bool = False,
125 |     admin_key: Optional[str] = None,
126 |     host: Optional[str] = None,
127 | ) -> ThreadingHTTPServer:
128 |     holder: Dict[str, ThreadingHTTPServer] = {}
129 |
130 |     def quit_() -> None:
131 |         monitor.stop()
132 |         holder["srv"].shutdown()
133 |         if on_quit:

```

```
134 |     on_quit()
135 |
136 |     bind = host or ("0.0.0.0" if server_mode else "127.0.0.1")
137 |     srv = ThreadingHTTPServer((bind, port), make_handler(monitor, port, quit_, server_mode, admin_key))
138 |     srv.daemon_threads = True
139 |     holder["srv"] = srv
140 |     return srv
=== END FILE: vero_status/server.py ===
```

```
=== FILE: vero_status/sessions.py (293 lines) ===
```

```

1 | """Your own Vero sessions (Vero CLI in a terminal, Vero in VS Code): latency and the model in use.
2 |
3 | How: Vero writes every task's event log to `tasks/<taskId>/ui_messages.json` (integration guide 3.5):
4 | a JSON list of events with `ts` (ms), `type`/'say' and `modelInfo`. Each model request starts with a
5 | `say: "api_req_started"` event; the first later event is the model's first response. The difference
6 | is the request latency, measured with Vero's own clock. Nothing is injected into Vero.
7 |
8 | Privacy: only timestamps, event kinds, modelInfo and the token counts inside `api_req_started` are read.
9 | Prompt and answer text are never kept. The files are only read, never changed.
10 | """
11 |
12 | from __future__ import annotations
13 |
14 | import glob
15 | import json
16 | import os
17 | import re
18 | import subprocess
19 | import sys
20 | import threading
21 | import time
22 | from dataclasses import dataclass, field
23 | from pathlib import Path
24 | from typing import Any, Dict, Iterable, List, Optional, Sequence, Set, Tuple
25 |
26 | from .vero import NO_WINDOW, PROMPT
27 |
28 | OPEN_WINDOW_S = 10 * 60_000 # a session with activity in the last 10 minutes counts as open
29 | SCAN_EVERY_S = 15
30 | MAX_TASK_AGE_S = 7 * 86400
31 | MAX_LATENCY_S = 30 * 60_000 # longer gaps are pauses (waiting for the user), not latency
32 | NOT_RESPONSES = {"api_req_started", "api_req_retried", "api_req_finished"}
33 |
34 |
35 | @dataclass(frozen=True)
36 | class Request:
37 |     start: float_000 # epoch seconds
38 |     seconds: float_000 # time to the model's first response
39 |     tokens_in: Optional[int]
40 |     tokens_out: Optional[int]
41 |     failed: bool
42 |
43 |
44 | @dataclass
45 | class TaskSummary:
46 |     task_id: str
47 |     source: str_000 # "CLI" or "VS Code"
48 |     first_prompt_is_probe: bool
49 |     model: Optional[str] = None
50 |     provider: Optional[str] = None
51 |     last_activity: float = 0.0
52 |     waiting_since: Optional[float] = None_000 # a request is running right now
53 |     requests: List[Request] = field(default_factory=list)
54 |
55 |
56 | def _int(value: Any) -> Optional[int]:
57 |     return value if isinstance(value, int) and not isinstance(value, bool) else None
58 |
59 |
60 | def parse_ui_messages(events: Sequence[Any], task_id: str, source: str, now: Optional[float] = None) -> TaskSummary:
61 |     """Pure: the event list of one task -> its model and request latencies."""
62 |     evs = sorted((e for e in events if isinstance(e, dict) and isinstance(e.get("ts"), (int, float))), key=lambda e: e["ts"])
63 |     first_text = next((str(e.get("text") or "") for e in evs if e.get("say") == "task"), "")
64 |     summary = TaskSummary(task_id, source, first_prompt_is_probe=first_text.strip() == PROMPT)
65 |     now = time.time() if now is None else now
66 |     for i, ev in enumerate(evs):

```

```

67 |         info = ev.get("modelInfo")
68 |         if isinstance(info, dict):
69 |             summary.model = info.get("modelId") or summary.model
70 |             summary.provider = info.get("providerId") or summary.provider
71 |             summary.last_activity = max(summary.last_activity, ev["ts"] / 1000)
72 |             if ev.get("say") != "api_req_started":
73 |                 continue
74 |             start = ev["ts"] / 1000
75 |             nxt = next((e for e in evs[i + 1 :] if e.get("say") not in NOT_RESPONSES), None)
76 |             if nxt is None:
77 |                 if now - start < MAX_LATENCY_S:
78 |                     summary.waiting_since = start
79 |                     continue
80 |             seconds = nxt["ts"] / 1000 - start
81 |             if not 0 <= seconds <= MAX_LATENCY_S:
82 |                 continue
83 |             tokens: Dict[str, Any] = {}
84 |             try: # only token counts; the "request" field (full prompt) is dropped immediately
85 |                 info_text = json.loads(ev.get("text") or "{}")
86 |                 if isinstance(info_text, dict):
87 |                     tokens = {k: info_text.get(k) for k in ("tokensIn", "tokensOut")}
88 |             except (TypeError, ValueError):
89 |                 pass
90 |             failed = nxt.get("say") in ("api_req_failed", "error") or nxt.get("ask") == "api_req_failed"
91 |             summary.requests.append(
92 |                 Request(start, round(seconds, 2), _int(tokens.get("tokensIn")), _int(tokens.get("tokensOut")), failed)
93 |             )
94 |             return summary
95 |
96 |
97 | def percentile(values: Sequence[float], pct: float) -> Optional[float]:
98 |     if not values:
99 |         return None
100 |     s = sorted(values)
101 |     k = (len(s) - 1) * pct / 100
102 |     lo, hi = int(k), min(int(k) + 1, len(s) - 1)
103 |     return round(s[lo] + (s[hi] - s[lo]) * (k - lo), 1)
104 |
105 |
106 | # ---- where Vero keeps its task logs -----
107 |
108 |
109 | def default_roots() -> List[Tuple[str, str]]:
110 |     """(glob pattern of a `tasks` folder, source label). CLI path measured; VS Code paths follow the
111 |     VS Code extension storage convention and are matched by an extension folder containing 'vero'."""
112 |     home = Path.home()
113 |     roots = [(str(home / ".vero" / "data" / "tasks"), "CLI")]
114 |     code_storage = [
115 |         Path(os.environ.get("APPDATA", home / "AppData" / "Roaming")) / "Code" / "User" / "globalStorage",
116 |         home / ".config" / "Code" / "User" / "globalStorage",
117 |         home / "Library" / "Application Support" / "Code" / "User" / "globalStorage",
118 |         home / ".vscode-server" / "data" / "User" / "globalStorage", # VS Code Remote-SSH on a Linux VM
119 |         home / ".vscode-server-insiders" / "data" / "User" / "globalStorage",
120 |     ]
121 |     for base in code_storage:
122 |         roots.append((str(base / "*vero*" / "tasks"), "VS Code"))
123 |         roots.append((str(base / "*Vero*" / "tasks"), "VS Code"))
124 |     return roots
125 |
126 |
127 | def find_task_files(roots: Iterable[Tuple[str, str]], now: float) -> List[Tuple[Path, str]]:
128 |     seen = set()
129 |     found: List[Tuple[Path, str]] = []
130 |     for pattern, source in roots:
131 |         for tasks_dir in glob.glob(pattern):
132 |             for f in glob.glob(os.path.join(glob.escape(tasks_dir), "*", "ui_messages.json")):
133 |                 p = Path(f)

```

```

134 | #####try:
135 | #####if now - p.stat().st_mtime > MAX_TASK_AGE_S or p in seen:
136 | #####continue
137 | #####except OSError:
138 | #####continue
139 | #####seen.add(p)
140 | #####found.append((p, source))
141 | #####return found
142 |
143 |
144 | # ---- is a Vero CLI open right now? -----
145 |
146 | _TOKEN = re.compile(r'("[^"]*"|'\"'\"')\S+')
147 | _VERO_SCRIPT = re.compile(r'^([\\\/])?([\\\/])?vero([\\\/])\.js$|$', re.I)
148 | VERO_EXES = {"vero", "vero.cmd", "vero.exe"}
149 | NODE_EXES = {"node", "node.exe"}
150 | ADMIN_COMMANDS = {"version", "--version", "-v", "config", "history", "h", "auth", "mcp", "token", "update", "dev", "--help", "-h"}
151 |
152 |
153 | def is_interactive_vero(cmdline: str, ignore: Sequence[str]) -> bool:
154 |     """True for a user's Vero CLI: the program is vero.cmd/.exe, or node running a script in a `vero`
155 |     package. Not our own checks (ignore list), not admin calls like `vero version`."""
156 |     tokens = [t.strip(' ') for t in _TOKEN.findall(cmdline or "")]
157 |     if not tokens:
158 |         return False
159 |     exe = re.split(r"[\\\/]", tokens[0])[-1].lower()
160 |     if exe in VERO_EXES:
161 |         args = tokens[1:]
162 |     elif exe in NODE_EXES and len(tokens) > 1 and _VERO_SCRIPT.search(tokens[1]):
163 |         args = tokens[2:]
164 |     else:
165 |         return False
166 |     low = cmdline.lower()
167 |     if any(x and x.lower() in low for x in ignore):
168 |         return False
169 |     return not (args and args[0].lower() in ADMIN_COMMANDS)
170 |
171 |
172 | def process_command_lines() -> List[str]:
173 |     """Command lines of node / vero processes. Windows: one hidden PowerShell call."""
174 |     try:
175 |         if sys.platform == "win32":
176 |             ps = (
177 |                 Get-CimInstance Win32_Process -Filter \"Name='node.exe' OR Name='vero.exe'\" | ForEach-Object { $_.CommandLine }
178 |             )
179 |             r = subprocess.run(
180 |                 ["powershell", "-NoProfile", "-NonInteractive", "-Command", ps],
181 |                 capture_output=True,
182 |                 text=True,
183 |                 timeout=15,
184 |                 creationflags=NO_WINDOW,
185 |                 check=False,
186 |             )
187 |         else:
188 |             # only your own processes: on a shared VM another user's Vero is not "your" open session
189 |             r = subprocess.run(
190 |                 ["ps", "-U", str(os.getuid()), "-o", "args="], capture_output=True, text=True, timeout=10, check=False
191 |             )
192 |             return [ln.strip() for ln in r.stdout.splitlines() if ln.strip()]
193 |         except (OSError, subprocess.SubprocessError):
194 |             return []
195 |
196 |
197 | # ---- the scanner -----
198 |
199 |
200 | class Scanner:

```

```

201 | """Re-reads changed task logs at most every SCAN_EVERY_S seconds (cached by file mtime)."""
202 |
203 | def __init__(
204 |     self,
205 |     roots: Optional[List[Tuple[str, str]]] = None,
206 |     ignore_processes: Sequence[str] = (),
207 |     list_processes: Any = process_command_lines,
208 | ) -> None:
209 |     self.roots = roots if roots is not None else default_roots()
210 |     self.ignore_processes = list(ignore_processes)
211 |     self.list_processes = list_processes
212 |     self._cache: Dict[Path, Tuple[float, TaskSummary]] = {}
213 |     self._last_scan = 0.0
214 |     self._result: Dict[str, Any] = {}
215 |     self._lock = threading.Lock()
216 |
217 | @property
218 | def result(self) -> Dict[str, Any]:
219 |     return self._result
220 |
221 | def state(self, own_task_ids: Iterable[str] = (), now: Optional[float] = None, force: bool = False) -> Dict[str, Any]:
222 |     now = time.time() if now is None else now
223 |     with self._lock:
224 |         if not force and self._result and now - self._last_scan < SCAN_EVERY_S:
225 |             return self._result
226 |         self._last_scan = now
227 |         self._result = self._scan(set(own_task_ids), now)
228 |         return self._result
229 |
230 | def _scan(self, own: Set[str], now: float) -> Dict[str, Any]:
231 |     tasks: List[TaskSummary] = []
232 |     live = set()
233 |     for path, source in find_task_files(self.roots, now):
234 |         live.add(path)
235 |         try:
236 |             mtime = path.stat().st_mtime
237 |             cached = self._cache.get(path)
238 |             if cached and cached[0] == mtime:
239 |                 summary = cached[1]
240 |             else:
241 |                 data = json.loads(path.read_text(encoding="utf-8"))
242 |                 summary = parse_ui_messages(data if isinstance(data, list) else [], path.parent.name, source, now)
243 |                 self._cache[path] = (mtime, summary)
244 |             except (OSError, ValueError):
245 |                 continue
246 |             if summary.task_id in own or summary.first_prompt_is_probe:
247 |                 continue  # our own availability checks are not your sessions
248 |             tasks.append(summary)
249 |             self._cache = {p: v for p, v in self._cache.items() if p in live}
250 |             return summarize(tasks, self._cli_open(), now, [r for r, _ in self.roots])
251 |
252 | def _cli_open(self) -> bool:
253 |     return any(is_interactive_vero(c, self.ignore_processes) for c in self.list_processes())
254 |
255 |
256 | def summarize(tasks: List[TaskSummary], cli_open: bool, now: float, roots: List[str]) -> Dict[str, Any]:
257 |     recent = sorted(tasks, key=lambda t: t.last_activity, reverse=True)
258 |     active = [t for t in recent if now - t.last_activity < OPEN_WINDOW_S or t.waiting_since]
259 |     current = active[0] if active else None  # an idle open CLI shows Vero's configured model instead
260 |     reqs = sorted([(r, t) for t in tasks for r in t.requests], key=lambda x: x[0].start)
261 |
262 | def window(seconds: float) -> Dict[str, Any]:
263 |     vals = [r.seconds for r, _ in reqs if now - r.start <= seconds and not r.failed]
264 |     return {"n": len(vals), "p50": percentile(vals, 50), "p95": percentile(vals, 95)}
265 |
266 | return {
267 |     "open": bool(active) or cli_open,

```

```

268 |         "cli_process": cli_open,
269 |         "current": None
270 |     if current is None
271 |     else {
272 |         "source": current.source,
273 |         "model": current.model,
274 |         "provider": current.provider,
275 |         "last_activity": current.last_activity,
276 |         "waiting_since": current.waiting_since,
277 |     },
278 |     "latency": {"hour": window(3600), "day": window(86400)},
279 |     "recent": [
280 |         {
281 |             "t": r.start,
282 |             "s": r.seconds,
283 |             "failed": r.failed,
284 |             "source": t.source,
285 |             "model": t.model,
286 |             "tokens_in": r.tokens_in,
287 |             "tokens_out": r.tokens_out,
288 |         }
289 |         for r, t in reqs[-40:]
290 |     ],
291 |     "tasks_seen": len(tasks),
292 |     "searched": roots,
293 | }
=== END FILE: vero_status/sessions.py ===

```

```

=== FILE: vero_status/settings.py (65 lines) ===
1 | """User settings, changed from the dashboard and kept in data/settings.json."""
2 |
3 | from __future__ import annotations
4 |
5 | import json
6 | from dataclasses import asdict, dataclass, replace
7 | from pathlib import Path
8 | from typing import Any, Dict
9 |
10 | INTERVALS = (0, 5, 15, 30, 60)  # minutes; 0 = only when "Check now" is clicked
11 | CHEAP_MODEL = "us.anthropic.claude-haiku-4-5-20251001-v1:0"
12 |
13 |
14 | class SettingsError(ValueError):
15 |     """pass"""
16 |
17 |
18 | @dataclass(frozen=True)
19 | class Settings:
20 |     """vero_path: str = ""  # empty = find it automatically
21 |     check_model: str = CHEAP_MODEL  # empty = Vero's own configured model
22 |     interval_minutes: int = 15
23 |     timeout_seconds: int = 120
24 |     port: int = 8767
25 |     track_sessions: bool = True  # read your own Vero sessions (CLI, VS Code) for latency and the model in use
26 |
27 |     def merged(self, changes: Dict[str, Any]) -> Settings:
28 |         """A validated copy with the given changes (unknown keys are rejected)."""
29 |         allowed = set(asdict(self))
30 |         unknown = set(changes) - allowed
31 |         if unknown:
32 |             raise SettingsError(f"unknown setting(s): {' '.join(sorted(unknown))}")
33 |         new = replace(self, **changes)
34 |         new.validate()
35 |         return new
36 |
37 |     def validate(self) -> None:
38 |         if not isinstance(self.vero_path, str) or not isinstance(self.check_model, str):
39 |             raise SettingsError("vero_path and check_model must be text")
40 |         if len(self.check_model) > 200 or any(c.isspace() for c in self.check_model):
41 |             raise SettingsError("the model id must be one word without spaces")
42 |         if self.interval_minutes not in INTERVALS:
43 |             raise SettingsError(f"interval must be one of {INTERVALS} minutes")
44 |         if not isinstance(self.timeout_seconds, int) or not 30 <= self.timeout_seconds <= 600:
45 |             raise SettingsError("timeout must be between 30 and 600 seconds")
46 |         if not isinstance(self.track_sessions, bool):
47 |             raise SettingsError("track_sessions must be true or false")
48 |         if not isinstance(self.port, int) or not 1024 <= self.port <= 65535:
49 |             raise SettingsError("port must be between 1024 and 65535")
50 |
51 |
52 | def load(path: Path) -> Settings:
53 |     """Settings from the file, or defaults. A broken file never stops the program."""
54 |     try:
55 |         raw = json.loads(path.read_text(encoding="utf-8-sig"))
56 |         return Settings().merged({k: v for k, v in raw.items() if k in asdict(Settings())})
57 |     except (OSError, ValueError, TypeError, AttributeError):
58 |         return Settings()
59 |
60 |
61 | def save(path: Path, settings: Settings) -> None:
62 |     path.parent.mkdir(parents=True, exist_ok=True)
63 |     tmp = path.with_suffix(".tmp")
64 |     tmp.write_text(json.dumps(asdict(settings), indent=2) + "\n", encoding="utf-8")
65 |     tmp.replace(path)
=== END FILE: vero_status/settings.py ===

```



```

67 |         found = shutil.which(candidate) or (candidate if Path(candidate).is_file() else None)
68 |         if found:
69 |             return found
70 |         return next((str(p) for p in npm_locations(Path.home()) if p.is_file()), None)
71 |
72 |
73 |     def child_env(exe: str) -> Dict[str, str]:
74 |         """`vero` is a node script (`#!/usr/bin/env node`); with nvm, node sits next to it but may not be on
75 |         the PATH of a service. Put the program's own folder first so its node is found."""
76 |         env = dict(os.environ)
77 |         folder = os.path.dirname(exe)
78 |         if folder:
79 |             env["PATH"] = folder + os.pathsep + env.get("PATH", "")
80 |         return env
81 |
82 |
83 |     def _kill_tree(proc: subprocess.Popen[bytes]) -> None:
84 |         with contextlib.suppress(OSError):
85 |             if sys.platform == "win32":
86 |                 subprocess.run(
87 |                     ["taskkill", "/T", "/F", "/PID", str(proc.pid)], capture_output=True, creationflags=NO_WINDOW, check=False
88 |                 )
89 |             else:
90 |                 os.killpg(proc.pid, signal.SIGKILL)
91 |         with contextlib.suppress(OSError):
92 |             proc.kill()
93 |
94 |
95 |     def run(args: Sequence[str], timeout_s: float, cwd: Optional[str] = None) -> Proc:
96 |         """Start a process without a window, collect output, kill the whole tree on timeout."""
97 |         t0 = time.perf_counter()
98 |         try:
99 |             proc = subprocess.Popen(
100 |                 list(args),
101 |                 stdin=subprocess.DEVNULL,
102 |                 stdout=subprocess.PIPE,
103 |                 stderr=subprocess.PIPE,
104 |                 cwd=cwd,
105 |                 env=child_env(args[0]),
106 |                 creationflags=NO_WINDOW,
107 |                 start_new_session=os.name != "nt",
108 |             )
109 |         except OSError as e:
110 |             return Proc(False, None, "", f"cannot start {Path(args[0]).name}: {e}", 0.0, False)
111 |         stdout, stderr = proc.stdout, proc.stderr
112 |         if stdout is None or stderr is None:
113 |             _kill_tree(proc)
114 |             return Proc(False, None, "", "no output pipes", 0.0, False)
115 |         out: List[bytes] = []
116 |         err: List[bytes] = []
117 |         readers = [
118 |             threading.Thread(target=lambda: out.append(stdout.read()), daemon=True),
119 |             threading.Thread(target=lambda: err.append(stderr.read()), daemon=True),
120 |         ]
121 |         for r in readers:
122 |             r.start()
123 |         timed_out = False
124 |         try:
125 |             proc.wait(timeout=timeout_s)
126 |         except subprocess.TimeoutExpired:
127 |             timed_out = True
128 |             _kill_tree(proc)
129 |             proc.wait()
130 |         seconds = time.perf_counter() - t0
131 |         for r in readers:
132 |             r.join(timeout=5)
133 |         stdout.close()

```

```

134 |     stderr.close()
135 |     return Proc(
136 |         True,
137 |         proc.returncode,
138 |         b"".join(out).decode("utf-8", "replace"),
139 |         b"".join(err).decode("utf-8", "replace"),
140 |         seconds,
141 |         timed_out,
142 |     )
143 |
144 |
145 | # ---- pure parsers -----
146 |
147 |
148 | def parse_version(text: str) -> str:
149 |     first = (text.strip().splitlines() or [""])[0]
150 |     return clean(first)[:60]
151 |
152 |
153 | CONFIG_KEYS = {
154 |     "actModeApiModelId": "model",
155 |     "actModeApiProvider": "provider",
156 |     "awsRegion": "region",
157 |     "planModeApiModelId": "plan_model",
158 | }
159 |
160 |
161 | def parse_config(text: str) -> Dict[str, str]:
162 |     """`vero config` output -> {model, provider, region, plan_model}. Accepts JSON or `key : value` lines."""
163 |     found: Dict[str, str] = {}
164 |     try:
165 |         data = json.loads(text)
166 |         if isinstance(data, dict):
167 |             for key, name in CONFIG_KEYS.items():
168 |                 if isinstance(data.get(key), (str, int, float)):
169 |                     found[name] = str(data[key])
170 |             return found
171 |         except json.JSONDecodeError:
172 |             pass
173 |         for line in text.splitlines():
174 |             m = re.match(r"^\s*['\"]?(\\w+)?(\\\"|\\')?\\s*[:=]\\s*['\"]?(\\^\\\"|,)+[\\\"']?\\.\\s*$", line)
175 |             if m and m.group(1) in CONFIG_KEYS:
176 |                 found[CONFIG_KEYS[m.group(1)]] = m.group(2).strip()
177 |         return found
178 |
179 |
180 | @dataclass(frozen=True)
181 | class TaskStream:
182 |     answer: Optional[str]
183 |     provider: Optional[str]
184 |     model: Optional[str]
185 |     error: Optional[str]
186 |     events: int
187 |     task_id: Optional[str] = None
188 |
189 |
190 | def parse_task_stream(lines: Iterable[str]) -> TaskStream:
191 |     answer = provider = model = error = task_id = None
192 |     events = 0
193 |     for raw in lines:
194 |         raw = raw.strip()
195 |         if not raw.startswith("{"):
196 |             continue
197 |         try:
198 |             ev = json.loads(raw)
199 |         except json.JSONDecodeError:
200 |             continue

```

```

201 |         if not isinstance(ev, dict):
202 |             continue
203 |         events += 1
204 |         if ev.get("type") == "task_started" and ev.get("taskId") is not None:
205 |             task_id = str(ev["taskId"])
206 |             info = ev.get("modelInfo")
207 |             if isinstance(info, dict):
208 |                 provider = info.get("providerId") or provider
209 |                 model = info.get("modelId") or model
210 |                 if ev.get("partial") is True:
211 |                     continue
212 |                 if ev.get("say") == "completion_result":
213 |                     answer = str(ev.get("text") or "")
214 |                     elif ev.get("say") in ("error", "api_req_failed") or ev.get("type") == "error":
215 |                         error = clean(str(ev.get("text") or ev.get("message") or "error"))
216 |                     return TaskStream(answer, provider, model, error, events, task_id)
217 |
218 |
219 | def task_args(exe: str, model: str, timeout_s: int, workdir: Path, prompt: str = PROMPT) -> List[str]:
220 |     args = [exe, "task", "--json", "-t", str(timeout_s), "-c", str(workdir)]
221 |     if model:
222 |         args += ["-m", model]
223 |     return [*args, prompt]
224 |
225 | # never --yolo: the probe needs no file or shell access
226 |
227 |
228 |
229 |
230 |
231 |
232 |
233 |
234 |
235 |
236 |
237 |
238 |
239 |
240 |
241 |
242 |
243 |
244 |
245 |
246 |
247 |
248 |
249 |
250 |
251 |
252 |
253 |
254 |
255 |
256 |
257 |
258 |
259 |
260 |
261 |
262 |
263 |
264 |
265 |
266 |
267 |
268 |
269 |
270 |
271 |
272 |
273 |
274 |
275 |
276 |
277 |
278 |
279 |
280 |
281 |
282 |
283 |
284 |
285 |
286 |
287 |
288 |
289 |
290 |
291 |
292 |
293 |
294 |
295 |
296 |
297 |
298 |
299 |
300 |
301 |
302 |
303 |
304 |
305 |
306 |
307 |
308 |
309 |
310 |
311 |
312 |
313 |
314 |
315 |
316 |
317 |
318 |
319 |
320 |
321 |
322 |
323 |
324 |
325 |
326 |
327 |
328 |
329 |
330 |
331 |
332 |
333 |
334 |
335 |
336 |
337 |
338 |
339 |
340 |
341 |
342 |
343 |
344 |
345 |
346 |
347 |
348 |
349 |
350 |
351 |
352 |
353 |
354 |
355 |
356 |
357 |
358 |
359 |
360 |
361 |
362 |
363 |
364 |
365 |
366 |
367 |
368 |
369 |
370 |
371 |
372 |
373 |
374 |
375 |
376 |
377 |
378 |
379 |
380 |
381 |
382 |
383 |
384 |
385 |
386 |
387 |
388 |
389 |
390 |
391 |
392 |
393 |
394 |
395 |
396 |
397 |
398 |
399 |
400 |
401 |
402 |
403 |
404 |
405 |
406 |
407 |
408 |
409 |
410 |
411 |
412 |
413 |
414 |
415 |
416 |
417 |
418 |
419 |
420 |
421 |
422 |
423 |
424 |
425 |
426 |
427 |
428 |
429 |
430 |
431 |
432 |
433 |
434 |
435 |
436 |
437 |
438 |
439 |
440 |
441 |
442 |
443 |
444 |
445 |
446 |
447 |
448 |
449 |
450 |
451 |
452 |
453 |
454 |
455 |
456 |
457 |
458 |
459 |
460 |
461 |
462 |
463 |
464 |
465 |
466 |
467 |
468 |
469 |
470 |
471 |
472 |
473 |
474 |
475 |
476 |
477 |
478 |
479 |
480 |
481 |
482 |
483 |
484 |
485 |
486 |
487 |
488 |
489 |
490 |
491 |
492 |
493 |
494 |
495 |
496 |
497 |
498 |
499 |
500 |
501 |
502 |
503 |
504 |
505 |
506 |
507 |
508 |
509 |
510 |
511 |
512 |
513 |
514 |
515 |
516 |
517 |
518 |
519 |
520 |
521 |
522 |
523 |
524 |
525 |
526 |
527 |
528 |
529 |
530 |
531 |
532 |
533 |
534 |
535 |
536 |
537 |
538 |
539 |
540 |
541 |
542 |
543 |
544 |
545 |
546 |
547 |
548 |
549 |
550 |
551 |
552 |
553 |
554 |
555 |
556 |
557 |
558 |
559 |
560 |
561 |
562 |
563 |
564 |
565 |
566 |
567 |
568 |
569 |
570 |
571 |
572 |
573 |
574 |
575 |
576 |
577 |
578 |
579 |
580 |
581 |
582 |
583 |
584 |
585 |
586 |
587 |
588 |
589 |
590 |
591 |
592 |
593 |
594 |
595 |
596 |
597 |
598 |
599 |
600 |
601 |
602 |
603 |
604 |
605 |
606 |
607 |
608 |
609 |
610 |
611 |
612 |
613 |
614 |
615 |
616 |
617 |
618 |
619 |
620 |
621 |
622 |
623 |
624 |
625 |
626 |
627 |
628 |
629 |
630 |
631 |
632 |
633 |
634 |
635 |
636 |
637 |
638 |
639 |
640 |
641 |
642 |
643 |
644 |
645 |
646 |
647 |
648 |
649 |
650 |
651 |
652 |
653 |
654 |
655 |
656 |
657 |
658 |
659 |
660 |
661 |
662 |
663 |
664 |
665 |
666 |
667 |
668 |
669 |
670 |
671 |
672 |
673 |
674 |
675 |
676 |
677 |
678 |
679 |
680 |
681 |
682 |
683 |
684 |
685 |
686 |
687 |
688 |
689 |
690 |
691 |
692 |
693 |
694 |
695 |
696 |
697 |
698 |
699 |
700 |
701 |
702 |
703 |
704 |
705 |
706 |
707 |
708 |
709 |
710 |
711 |
712 |
713 |
714 |
715 |
716 |
717 |
718 |
719 |
720 |
721 |
722 |
723 |
724 |
725 |
726 |
727 |
728 |
729 |
730 |
731 |
732 |
733 |
734 |
735 |
736 |
737 |
738 |
739 |
740 |
741 |
742 |
743 |
744 |
745 |
746 |
747 |
748 |
749 |
750 |
751 |
752 |
753 |
754 |
755 |
756 |
757 |
758 |
759 |
760 |
761 |
762 |
763 |
764 |
765 |
766 |
767 |
768 |
769 |
770 |
771 |
772 |
773 |
774 |
775 |
776 |
777 |
778 |
779 |
780 |
781 |
782 |
783 |
784 |
785 |
786 |
787 |
788 |
789 |
790 |
791 |
792 |
793 |
794 |
795 |
796 |
797 |
798 |
799 |
800 |
801 |
802 |
803 |
804 |
805 |
806 |
807 |
808 |
809 |
810 |
811 |
812 |
813 |
814 |
815 |
816 |
817 |
818 |
819 |
820 |
821 |
822 |
823 |
824 |
825 |
826 |
827 |
828 |
829 |
830 |
831 |
832 |
833 |
834 |
835 |
836 |
837 |
838 |
839 |
840 |
841 |
842 |
843 |
844 |
845 |
846 |
847 |
848 |
849 |
850 |
851 |
852 |
853 |
854 |
855 |
856 |
857 |
858 |
859 |
860 |
861 |
862 |
863 |
864 |
865 |
866 |
867 |
868 |
869 |
870 |
871 |
872 |
873 |
874 |
875 |
876 |
877 |
878 |
879 |
880 |
881 |
882 |
883 |
884 |
885 |
886 |
887 |
888 |
889 |
890 |
891 |
892 |
893 |
894 |
895 |
896 |
897 |
898 |
899 |
900 |
901 |
902 |
903 |
904 |
905 |
906 |
907 |
908 |
909 |
910 |
911 |
912 |
913 |
914 |
915 |
916 |
917 |
918 |
919 |
920 |
921 |
922 |
923 |
924 |
925 |
926 |
927 |
928 |
929 |
930 |
931 |
932 |
933 |
934 |
935 |
936 |
937 |
938 |
939 |
940 |
941 |
942 |
943 |
944 |
945 |
946 |
947 |
948 |
949 |
950 |
951 |
952 |
953 |
954 |
955 |
956 |
957 |
958 |
959 |
960 |
961 |
962 |
963 |
964 |
965 |
966 |
967 |
968 |
969 |
970 |
971 |
972 |
973 |
974 |
975 |
976 |
977 |
978 |
979 |
980 |
981 |
982 |
983 |
984 |
985 |
986 |
987 |
988 |
989 |
990 |
991 |
992 |
993 |
994 |
995 |
996 |
997 |
998 |
999 |
1000 |

```

=== FILE: vero_status/web/index.html (383 lines) ===

```

1 | <!doctype html>
2 | <html lang="en">
3 | <head>
4 | <meta charset="utf-8">
5 | <meta name="viewport" content="width=device-width, initial-scale=1">
6 | <title>Vero Status</title>
7 | <link rel="icon" id="favicon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 16 16'%3E%3Ccircle cx='8' cy='8' r='7' fill='%238a94a3'/%3E%3C/svg%3E">
8 | <style>
9 | :root{
10 |   --ink:#1b2230; --mut:#5d6878; --line:#dde2ea; --paper:#f4f6f9; --card:#ffffff; --brand:#0a6aa1;
11 |   --ok:#1f7a45; --ok-bg:#e8f4ed; --bad:#9b1c2e; --bad-bg:#f8e7ea; --warn:#a8500b; --idle:#8a94a3; --idle-bg:#eef1f5;
12 |   --sans:"Segoe UI","IBM Plex Sans",Roboto,Helvetica,Arial,sans-serif; --mono:Consolas,"IBM Plex Mono",ui-monospace,monospace;
13 | }
14 | @media (prefers-color-scheme: dark){:root{
15 |   --ink:#e6eaf0; --mut:#9aa5b4; --line:#2c3441; --paper:#10141a; --card:#171c24; --brand:#4fa8dc;
16 |   --ok:#4cc38a; --ok-bg:#132a1f; --bad:#f06b7f; --bad-bg:#321a20; --warn:#e39a4b; --idle:#6f7a89; --idle-bg:#1c222b;
17 | }}
18 | *{box-sizing:border-box}
19 | body{margin:0;background:var(--paper);color:var(--ink);font:15px/1.5 var(--sans)}
20 | .wrap{max-width:880px;margin:0 auto;padding:22px 20px 40px}
21 | header{display:flex;align-items:center;gap:12px;flex-wrap:wrap;margin-bottom:16px}
22 | header h1{font-size:20px;margin:0;font-weight:700}
23 | header .ver{color:var(--mut);font-size:13px}
24 | header .sp{flex:1}
25 | button{font:inherit;font-size:14px;border:1px solid var(--line);background:var(--card);color:var(--ink);border-radius:8px;padding:8px 14px;cursor:pointer}
26 | button.primary{background:var(--brand);border-color:var(--brand);color:#fff;font-weight:600}
27 | button.disabled{opacity:.55;cursor:default}
28 | button:focus-visible,input:focus-visible,select:focus-visible{outline:2px solid var(--brand);outline-offset:2px}
29 | .card{background:var(--card);border:1px solid var(--line);border-radius:12px;padding:18px 20px;margin-bottom:14px}
30 | .hero{display:flex;gap:20px;align-items:center;padding:26px 24px;background:var(--idle-bg)}
31 | .hero.ok{background:var(--ok-bg);border-color:color-mix(in srgb,var(--ok) 35%,var(--line))}
32 | .hero.bad{background:var(--bad-bg);border-color:color-mix(in srgb,var(--bad) 35%,var(--line))}
33 | .lamp{width:72px;height:72px;border-radius:50%;flex:none;display:flex;align-items:center;justify-content:center;color:#fff;font-size:36px;font-weight:800;background:var(--idle)}
34 | .ok .lamp{background:var(--ok)} .bad .lamp{background:var(--bad)}
35 | .checking .lamp{animation:pulse 1.2s ease-in-out infinite}
36 | @keyframes pulse{50%{opacity:.45}}
37 | @media (prefers-reduced-motion:reduce){.checking .lamp{animation:none}}
38 | .hero h2{margin:0;font-size:28px;line-height:1.2}
39 | .hero .meta{color:var(--mut);margin-top:4px}
40 | .hero .why{margin-top:8px;font-size:14px}
41 | .ok .why{color:var(--warn)} .bad .why{color:var(--bad)}
42 | .label{font-size:12px;text-transform:uppercase;letter-spacing:.06em;color:var(--mut);margin-bottom:6px}
43 | .model{display:grid;grid-template-columns:1fr 1fr;gap:18px}
44 | .model .name{font-size:22px;font-weight:700;line-height:1.25}
45 | .model .id{font-family:var(--mono);font-size:12px;color:var(--mut);overflow-wrap:anywhere;margin-top:2px}
46 | .model .sub{font-size:13px;color:var(--mut);margin-top:4px}
47 | .facts{display:grid;grid-template-columns:repeat(3,1fr);gap:14px}
48 | .facts .v{font-size:22px;font-weight:700;font-variant-numeric:tabular-nums}
49 | .dots{display:flex;gap:4px;flex-wrap:wrap;margin-top:4px}
50 | .live{display:flex;align-items:center;gap:12px;padding:12px 18px}
51 | .live .dot{width:12px;height:12px;border-radius:50%;background:var(--idle);flex:none}
52 | .live.on .dot{background:var(--ok);box-shadow:0 0 4px color-mix(in srgb,var(--ok) 22%,transparent)}
53 | .live .what{font-weight:600}
54 | .live .mdl{margin-left:auto;text-align:right}
55 | .live .mdl b{font-size:17px}
56 | .live .mdl .id{font-family:var(--mono);font-size:11.5px;color:var(--mut)}
57 | .bars{display:flex;align-items:flex-end;gap:2px;height:96px;margin-top:6px;border-bottom:1px solid var(--line);position:relative}
58 | .bars .b{flex:1;max-width:22px;min-height:2px;background:var(--brand);border-radius:4px 4px 0 0;position:relative;cursor:default}
59 | .bars .b.fail{background:var(--bad)}
60 | .bars .b:hover,.bars .b:focus{outline:2px solid var(--ink);outline-offset:1px}
61 | .bars .p50{position:absolute;left:0;right:0;border-top:1px dashed var(--mut);pointer-events:none}
62 | .bars .p50 span{position:absolute;right:0;top:-17px;font-size:11px;color:var(--mut);background:var(--card);padding:0 3px}
63 | .tip{position:fixed;pointer-events:none;background:var(--card);border:1px solid var(--line);border-radius:6px;padding:6px 9px;font-size:12.5px;box-shadow:0 4px 14px rgba(0,0,0,.15);display:none;z
+ | -index:5}
64 | .check{display:flex;align-items:center;gap:8px}
65 | .badge{font-size:12px;font-weight:600;padding:3px 10px;border-radius:999px;border:1px solid var(--line);color:var(--mut);background:var(--card)}

```

```

66 | .act{width:100%;border-collapse:collapse;font-size:13.5px;margin-top:4px}
67 | .act th,.act td{text-align:left;padding:6px 8px;border-bottom:1px solid var(--line);vertical-align:top}
68 | .act th{font-size:12px;color:var(--mut);font-weight:600}
69 | .act td.n{text-align:right;font-variant-numeric:tabular-nums;white-space:nowrap}
70 | .act .ok{color:var(--ok);font-weight:600} .act .bad{color:var(--bad);font-weight:600}
71 | .act .why{color:var(--mut)}
72 | .tw{overflow-x:auto}
73 | .check input{width:auto}
74 | .dots span{width:14px;height:14px;border-radius:4px;background:var(--idle)}
75 | .dots span.ok{background:var(--ok)} .dots span.bad{background:var(--bad)}
76 | .muted{color:var(--mut);font-size:13px}
77 | .settings{display:none}
78 | .settings.open{display:block}
79 | .form{display:grid;grid-template-columns:220px 1fr;gap:12px 16px;align-items:center}
80 | .form input,.form select{font:inherit;font-size:14px;padding:7px 10px;border:1px solid var(--line);border-radius:8px;background:var(--card);color:var(--ink);width:100%}
81 | .form .hint{grid-column:2;font-size:12.5px;color:var(--mut);margin-top:-6px}
82 | .row{display:flex;gap:10px;align-items:center;margin-top:14px}
83 | .msg{font-size:13px}
84 | .msg.err{color:var(--bad)} .msg.ok{color:var(--ok)}
85 | .overlay{position:fixed;inset:0;background:var(--paper);display:none;align-items:center;justify-content:center;text-align:center;padding:20px}
86 | .overlay.show{display:flex}
87 | .overlay h2{margin:0 0 8px}
88 | @media (max-width:640px){.live{flex-wrap:wrap}.live .mdl{margin-left:24px;text-align:left;width:100%}.model,.facts{grid-template-columns:1fr}.form{grid-template-columns:1fr}.form .hint{grid-column
+ n:1}
89 |   .hero{flex-direction:column;align-items:flex-start}.hero h2{font-size:23px}}
90 | </style>
91 | </head>
92 | <body>
93 | <div class="wrap">
94 |   <header>
95 |     <h1>Vero Status</h1><span class="ver" id="ver"></span>
96 |     <span class="sp"></span>
97 |     <button class="primary" id="checkBtn" type="button">Check now</button>
98 |     <span class="badge" id="modeBadge" hidden></span>
99 |     <button id="setBtn" type="button" aria-expanded="false" aria-controls="settings">Settings</button>
100 |     <button id="quitBtn" type="button">Quit</button>
101 |   </header>
102 |
103 |   <section class="card hero" id="hero" aria-live="polite">
104 |     <div class="lamp" id="lamp">?</div>
105 |     <div>
106 |       <h2 id="title">Starting...</h2>
107 |       <div class="meta" id="meta"></div>
108 |       <div class="why" id="why"></div>
109 |     </div>
110 |   </section>
111 |
112 |   <section class="card live" id="live" aria-live="polite">
113 |     <span class="dot"></span>
114 |     <div><div class="what" id="liveWhat">Looking for an open Vero...</div><div class="muted" id="liveSub"></div></div>
115 |     <div class="mdl" id="liveModel"></div>
116 |   </section>
117 |
118 |   <section class="card">
119 |     <div class="model">
120 |       <div>
121 |         <div class="label">Vero's model</div>
122 |         <div class="name" id="cfgName">—</div>
123 |         <div class="id" id="cfgId"></div>
124 |         <div class="sub" id="cfgSub"></div>
125 |       </div>
126 |       <div>
127 |         <div class="label" id="ansLabel">Answered the last check</div>
128 |         <div class="name" id="ansName">—</div>
129 |         <div class="id" id="ansId"></div>
130 |         <div class="sub" id="ansSub"></div>
131 |       </div>

```

```

132 |     </div>
133 | </section>
134 |
135 | <section class="card">
136 |     <div class="facts">
137 |         <div><div class="label">Available, last 24 h</div><div class="v" id="avail"></div><div class="muted" id="availSub"></div></div>
138 |         <div><div class="label">Last answer took</div><div class="v" id="took"></div><div class="muted">for "Reply with exactly: OK"</div></div>
139 |         <div><div class="label">Next check</div><div class="v" id="next"></div><div class="muted" id="every"></div></div>
140 |     </div>
141 |     <div class="label" style="margin-top:16px">Last checks (oldest → newest)</div>
142 |     <div class="dots" id="dots"></div>
143 | </section>
144 |
145 | <section class="card" id="actCard">
146 |     <div class="label">Recent activity</div>
147 |     <div class="tw"><table class="act" id="act"></table></div>
148 | </section>
149 |
150 | <section class="card" id="latCard">
151 |     <div class="label">Your Vero sessions (CLI and VS Code): response time</div>
152 |     <div class="facts">
153 |         <div><div class="label">Median, last hour</div><div class="v" id="latP50"></div><div class="muted" id="latN"></div></div>
154 |         <div><div class="label">Slowest 5 %, last hour</div><div class="v" id="latP95"></div><div class="muted">p95</div></div>
155 |         <div><div class="label">Last 24 h</div><div class="v" id="latDay"></div><div class="muted" id="latDayN"></div></div>
156 |     </div>
157 |     <div class="label" style="margin-top:14px">Last model requests (oldest → newest), seconds until the model started answering</div>
158 |     <div class="bars" id="latBars" role="img" aria-label="Response time of your last Vero model requests"></div>
159 |     <div class="muted" id="latNote" style="margin-top:8px"></div>
160 | </section>
161 |
162 | <section class="card settings" id="settings" aria-label="Settings">
163 |     <div class="label">Settings</div>
164 |     <div class="form">
165 |         <label for="sInterval">Check automatically</label>
166 |         <select id="sInterval">
167 |             <option value="0">Only when I click "Check now"</option>
168 |             <option value="5">Every 5 minutes</option>
169 |             <option value="15">Every 15 minutes</option>
170 |             <option value="30">Every 30 minutes</option>
171 |             <option value="60">Every hour</option>
172 |         </select>
173 |         <div class="hint">Each check asks Vero one tiny question (about 1 minute, 2 small model calls).</div>
174 |         <label for="sModel">Model used for the check</label>
175 |         <input id="sModel" spellcheck="false" placeholder="empty = Vero's own model">
176 |         <div class="hint">A cheap model keeps checks cheap. Leave empty to test the model Vero is set to.</div>
177 |         <label for="sTimeout">Give up after (seconds)</label>
178 |         <input id="sTimeout" type="number" min="30" max="600" step="10">
179 |         <label for="sPath">Vero CLI location</label>
180 |         <input id="sPath" spellcheck="false" placeholder="empty = find it automatically">
181 |         <div class="hint" id="found"></div>
182 |         <label for="sTrack">My own Vero sessions</label>
183 |         <label class="check"><input id="sTrack" type="checkbox"> Show the model and response time of Vero CLI / VS Code sessions</label>
184 |         <div class="hint">Reads only timestamps and model names from Vero's own task logs. Your prompts and answers are never read.</div>
185 |     </div>
186 |     <div class="row"><button class="primary" id="saveBtn" type="button">Save</button><span class="msg" id="saveMsg"></span></div>
187 | </section>
188 |
189 | <p class="muted" id="foot"></p>
190 | </div>
191 |
192 | <div class="tip" id="tip"></div>
193 | <div class="overlay" id="stopped"><div><h2>Vero Status is stopped</h2>
194 | <p class="muted">Double-click <b>Vero Status</b> to start it again. You can close this tab.</p></div></div>
195 |
196 | <script>
197 | (function(){
198 | "use strict";

```

```

199 | const $ = (id) => document.getElementById(id);
200 | let S = null, timer = null, settingsLoaded = false, offline = false;
201 |
202 | function prettyModel(id){
203 |   if (!id) return null;
204 |   const m = /claude-(opus|sonnet|haiku|fable)-(\d+)(?-(\d{1,2}))?(?!\\d)/i.exec(id);
205 |   if (m) return "Claude " + m[1][0].toUpperCase() + m[1].slice(1).toLowerCase() + " " + m[2] + (m[3] ? "." + m[3] : "");
206 |   return id.split(/[/:]/).filter(Boolean).slice(-1)[0] || id;
207 | }
208 | function ago(t){ const s = Math.max(0, Math.round(Date.now()/1000 - t));
209 |   if (s < 60) return s + " s ago"; if (s < 3600) return Math.round(s/60) + " min ago";
210 |   if (s < 86400) return Math.round(s/3600) + " h ago"; return Math.round(s/86400) + " d ago"; }
211 | const clock = (t) => new Date(t*1000).toLocaleTimeString(undefined, {hour:"2-digit", minute:"2-digit"});
212 | const day = (t) => new Date(t*1000).toLocaleString(undefined, {month:"short", day:"numeric", hour:"2-digit", minute:"2-digit"});
213 | function setIcon(color){ $("favicon").href = "data:image/svg+xml," + encodeURIComponent(
214 |   `<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 16 16'><circle cx='8' cy='8' r='7' fill='${color}'/></svg>`); }
215 |
216 | function render(){
217 |   const s = S, last = s.last;
218 |   $("ver").textContent = "v" + s.version;
219 |   const hero = $("hero");
220 |   let cls = "", title, icon = "?", meta = "", why = "";
221 |   if (s.checking && !last) { title = "Checking Vero..."; meta = "The first check takes about a minute."; }
222 |   else if (!last) { title = "Not checked yet"; meta = 'Click "Check now"'; }
223 |   else {
224 |     cls = last.available ? "ok" : "bad"; icon = last.available ? "✓" : "x";
225 |     title = last.available ? "Vero is available" : "Vero is not available";
226 |     meta = `since ${day(s.since)} · last check ${ago(last.time)}`;
227 |     if (s.checking) meta = "Checking again now... " + meta;
228 |     why = last.reason || "";
229 |   }
230 |   hero.className = "card hero " + cls + (s.checking ? " checking" : "");
231 |   $("lamp").textContent = icon; $("title").textContent = title; $("meta").textContent = meta; $("why").textContent = why;
232 |   document.title = (last ? (last.available ? "✓" : "x") : "") + "Vero Status";
233 |   setIcon(!last ? "#8a94a3" : last.available ? "#1f7a45" : "#9b1c2e");
234 |
235 |   const c = s.configured || {};
236 |   $("cfgName").textContent = prettyModel(c.model) || "unknown";
237 |   $("cfgId").textContent = c.model || "";
238 |   $("cfgSub").textContent = [c.provider && "provider " + c.provider, c.region].filter(Boolean).join(" · ") || "from vero config";
239 |   const asked = last && last.asked_model;
240 |   $("ansLabel").textContent = !last || last.available ? "Answered the last check" : "Asked in the last check: no answer";
241 |   $("ansName").textContent = prettyModel(asked) || "-";
242 |   $("ansId").textContent = asked || "";
243 |   $("ansSub").textContent = last ? [last.provider && "provider " + last.provider, last.vero_version].filter(Boolean).join(" · ") : "";
244 |
245 |   $("avail").textContent = s.availability_24h == null ? "-" : s.availability_24h + "%";
246 |   $("availSub").textContent = s.checks_24h ? `${s.checks_24h} check${s.checks_24h > 1 ? "s" : ""}` : "no checks yet";
247 |   $("took").textContent = last && last.available && last.seconds != null ? Math.round(last.seconds) + " s" : "-";
248 |   const iv = s.settings.interval_minutes;
249 |   $("every").textContent = iv ? `every ${iv} min` : "automatic checks are off";
250 |   tick();
251 |   $("dots").innerHTML = s.history.map(r => `<span class="${r.available ? "ok" : "bad"}" title="${day(r.time)}: ${r.available ? "available" : "not available"}${r.seconds != null ? ", " + Math.round(r.seconds) + " s" : ""}></span>`).join("")
+ d(r.seconds) + " s" : ""}></span>`);
252 |   | 'none yet</span>';
253 |   const server = s.mode === "server", wait = s.check_wait_s || 0;
254 |   $("checkBtn").disabled = s.checking || wait > 0;
255 |   $("checkBtn").textContent = s.checking ? "Checking..." : wait > 0 ? `Check now (in ${Math.ceil(wait / 60)} min)` : "Check now";
256 |   $("checkBtn").title = wait > 0 ? "Someone checked recently. On the shared server, manual checks are limited to one every 5 minutes." : "";
257 |   $("quitBtn").hidden = server;
258 |   $("setBtn").hidden = server && !s.admin;
259 |   if (server && !s.admin) $("settings").classList.remove("open");
260 |   $("modeBadge").hidden = !server;
261 |   $("modeBadge").textContent = s.admin ? "Team server · admin" : "Team server";
262 |   renderActivity(s.history);
263 |   $("found").textContent = s.settings.vero_found ? "Found: " + s.settings.vero_found : "Vero CLI not found automatically: enter its full path (in a terminal: " + (s.platform === "win32" ? "where_
+ vero" : "command -v vero") + ")";

```

```

264 |   ui$("foot").textContent = (server
265 |   ui$? `Shared status page for the team, no login. Vero is checked on ${s.host || "the server"} every ${iv || "-" } min. `
266 |   ui$: `Runs on ${s.host || "this computer"} only (127.0.0.1). `)
267 |   ui$+ "The check asks: \"Reply with exactly: OK\". No answers or secrets are stored.";
268 |   ui.renderSessions(s);
269 |   ui.if (!settingsLoaded) {
270 |     ui$("${sTrack").checked = !!s.settings.track_sessions;
271 |     ui$("${sInterval").value = String(iv); ui$("${sModel").value = s.settings.check_model;
272 |     ui$("${sTimeout").value = s.settings.timeout_seconds; ui$("${sPath").value = s.settings.vero_path; settingsLoaded = true;
273 |   }
274 | }
275 | const TRIG = {auto: "scheduled", manual: "Check now", start: "at start"};
276 | function renderActivity(hist){
277 |   ui$const rows = (hist || []).slice(-10).reverse();
278 |   ui$("act").innerHTML = rows.length
279 |   ui$? `<thead><tr><th>Time</th><th>Result</th><th>Model</th><th class=\`"\">Answer time</th><th>Started by</th><th>Note</th></tr></thead><tbody>"
280 |   ui$+ rows.map(r => `<tr><td>${esc(day(r.time))}</td><td class=\`${r.available ? "ok" : "bad"}>${r.available ? "available" : "not available"}</td>`
281 |   ui$+ `<td>${esc(prettyModel(r. asked_model) || "-")}</td><td class="n">${r.available && r.seconds != null ? Math.round(r.seconds) + " s" : "-"}</td>`
282 |   ui$+ `<td>${esc(TRIG[r.trigger] || r.trigger || "")}</td><td class="why">${esc(r.reason || "")}</td></tr>`).join("") + "</tbody>"
283 |   ui$: `<tbody><tr><td class="why">No checks yet.</td></tr></tbody>`;
284 | }
285 | const secs = (v) => v == null ? "-" : (v < 10 ? v.toFixed(1) : Math.round(v)) + " s";
286 | function renderSessions(s){
287 |   ui$const on = s.settings.track_sessions, x = s.sessions;
288 |   ui$("live").style.display = on ? "" : "none"; ui$("latCard").style.display = on ? "" : "none";
289 |   ui.if (!on) return;
290 |   ui.if (!x || !x.latency) { ui$("liveWhat").textContent = "Looking for an open Vero..."; return; }
291 |   ui$const cur = x.current, cfg = s.configured || {};
292 |   ui$const model = cur ? cur.model : (x.cli_process ? cfg.model : null);
293 |   ui$("live").className = "card live" + (x.open ? " on" : "");
294 |   ui$("liveWhat").textContent = !x.open ? "No Vero CLI or VS Code session open"
295 |   ui$: cur ? `Vero ${cur.source === "VS Code" ? "in VS Code" : "CLI"} is open` : "Vero CLI is open";
296 |   ui$("liveSub").textContent = !x.open ? "The model appears here while you use Vero."
297 |   ui$: cur && cur.waiting_since ? `waiting for the model for ${Math.round(Date.now()/1000 - cur.waiting_since)} s`
298 |   ui$: cur ? `last activity ${ago(cur.last_activity)}` : "idle; showing Vero's configured model";
299 |   ui$("liveModel").innerHTML = x.open && model
300 |   ui$: `<div class="label" style="margin:0">Current model</div><b>${esc(prettyModel(model))}</b><div class="id">${esc(model)}</div>` : "";
301 |   ui$const h = x.latency.hour, d = x.latency.day;
302 |   ui$("latP50").textContent = secs(h.p50); ui$("latP95").textContent = secs(h.p95);
303 |   ui$("latN").textContent = h.n ? `${h.n} request${h.n > 1 ? "s" : ""}` : "no requests";
304 |   ui$("latDay").textContent = secs(d.p50); ui$("latDayN").textContent = d.n ? `median of ${d.n} request${d.n > 1 ? "s" : ""}` : "no requests";
305 |   ui$const rs = x.recent || [], max = Math.max(1, ...rs.map(r => r.s));
306 |   ui$const p50 = h.p50 != null ? h.p50 : d.p50;
307 |   ui$("latBars").innerHTML = rs.length ? rs.map((r, i) =>
308 |     ui$`<div class="b${r.failed ? " fail" : ""}" tabindex="0" data-i="${i}" style="height:${Math.max(2, r.s / max * 92)}px"></div>`).join("")
309 |     ui$+ (p50 != null ? `<div class="p50" style="bottom:${p50 / max * 92}px"><span>median ${secs(p50)}</span></div>` : "")
310 |     ui$: `<span class="muted" style="align-self:center">No requests from your Vero sessions in the last 7 days.</span>`;
311 |   ui$("latBars").dataset.rows = JSON.stringify(rs);
312 |   ui$("latNote").textContent = "Measured from Vero's own task logs: time from each model request to the first response. "
313 |   ui$+ "The tiny test questions of this dashboard are not included." + (x.tasks_seen ? "" : " Looked in: " + (x.searched || []).join(" · "));
314 | }
315 | const esc = (v) => String(v == null ? "" : v).replace(/([<>])/g, c => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&quot;"}[c]));
316 | function showTip(e){
317 |   ui$const b = e.target.closest(".b"); const tip = ui$("tip");
318 |   ui.if (!b) { tip.style.display = "none"; return; }
319 |   ui$const r = JSON.parse(ui$("latBars").dataset.rows || "[[]]"+b.dataset.i); if (!r) return;
320 |   ui$tip.innerHTML = `<b>${secs(r.s)}</b><b>${r.failed ? " · failed" : ""}</b><br>${esc(day(r.t))} · ${esc(r.source)}<br>${esc(prettyModel(r.model) || "")}`
321 |   ui$+ (r.tokens_in != null ? `<br>${r.tokens_in} tokens in · ${r.tokens_out != null ? r.tokens_out : "?"} out` : "");
322 |   ui$tip.style.display = "block";
323 |   ui$const rect = b.getBoundingClientRect();
324 |   ui$tip.style.left = Math.min(rect.left, innerWidth - tip.offsetWidth - 8) + "px"; tip.style.top = (rect.top - tip.offsetHeight - 6) + "px";
325 | }
326 | ["mouseover", "focusin"].forEach(ev => ui$("latBars").addEventListener(ev, showTip));
327 | ["mouseleave", "focusout"].forEach(ev => ui$("latBars").addEventListener(ev, () => { ui$("tip").style.display = "none"; }));
328 |
329 | function tick(){
330 |   ui.if (!S) return;

```

```

331 |   const n = S.next_check;
332 |   if (S.checking) { $("next").textContent = "now"; return; }
333 |   if (!n) { $("next").textContent = "-"; return; }
334 |   const s = Math.max(0, Math.round(n - Date.now()/1000));
335 |   $("next").textContent = s < 60 ? `in ${s} s` : s < 3600 ? `in ${Math.ceil(s/60)} min` : "at " + clock(n);
336 | }
337 | let ADMIN = null;
338 | try {
339 |   const u = new URL(location.href), k = u.searchParams.get("admin");
340 |   if (k) { sessionStorage.setItem("vs.admin", k); u.searchParams.delete("admin"); history.replaceState(null, "", u.pathname + u.search); }
341 |   ADMIN = sessionStorage.getItem("vs.admin");
342 | } catch (e) {}
343 | const hdrs = (extra) => Object.assign(ADMIN ? {"X-Admin-Key": ADMIN} : {}, extra || {});
344 | async function load(){
345 |   try {
346 |     const r = await fetch("/api/state", {cache: "no-store", headers: hdrs()});
347 |     S = await r.json(); offline = false; render();
348 |   } catch (e) {
349 |     offline = true; $("hero").className = "card hero"; $("lamp").textContent = "!";
350 |     $("title").textContent = "Vero Status is not running"; $("meta").textContent = "Double-click Vero Status to start it.";
351 |   }
352 |   clearTimeout(timer); timer = setTimeout(load, S && S.checking ? 1500 : 5000);
353 | }
354 | async function post(path, body){
355 |   return fetch(path, {method: "POST", headers: hdrs({"Content-Type": "application/json"}), body: body ? JSON.stringify(body) : undefined});
356 | }
357 | $("checkBtn").addEventListener("click", async () => {
358 |   $("checkBtn").disabled = true;
359 |   const r = await post("/api/check");
360 |   if (r.status === 429) { const d = await r.json().catch(() => ({})); alert(d.error || "Checked recently, try again later."); }
361 |   load();
362 | });
363 | $("setBtn").addEventListener("click", () => { const p = $("settings"), open = !p.classList.contains("open");
364 |   p.classList.toggle("open", open); $("setBtn").setAttribute("aria-expanded", open); if (open) p.scrollIntoView({behavior: "smooth"}); });
365 | $("saveBtn").addEventListener("click", async () => {
366 |   const body = {interval_minutes: +$("sInterval").value, check_model: $("sModel").value.trim(),
367 |     timeout_seconds: Math.round(+$("sTimeout").value), vero_path: $("sPath").value.trim().replace(/^|"/g, ""),
368 |     track_sessions: $("sTrack").checked};
369 |   const r = await post("/api/settings", body), d = await r.json().catch(() => ({}));
370 |   const m = $("saveMsg"); m.className = "msg" + (r.ok ? "ok" : "err"); m.textContent = r.ok ? "Saved." : (d.error || "Not saved.");
371 |   if (r.ok) { settingsLoaded = false; load(); }
372 | });
373 | $("quitBtn").addEventListener("click", async () => {
374 |   if (!confirm("Stop Vero Status? Checks stop until you start it again.)) return;
375 |   try { await post("/api/quit"); } catch (e) {}
376 |   clearTimeout(timer); $("stopped").classList.add("show");
377 | });
378 | setInterval(tick, 1000);
379 | load();
380 | })();
381 | </script>
382 | </body>
383 | </html>
=== END FILE: vero_status/web/index.html ===

```

```
=== FILE: vero_status_server.py (16 lines) ===
1 | """Vero Status team server: one shared status page for everyone on the network, no login.
2 |
3 | python vero_status_server.py (port from data/settings.json, default 8767)
4 | python vero_status_server.py --port 8080
5 |
6 | The share link and the private admin link are printed and written to data/vero-status.log.
7 | """
8 |
9 | import sys
10 | from pathlib import Path
11 |
12 | sys.path.insert(0, str(Path(__file__).resolve().parent))
13 |
14 | from vero_status.app import serve_team
15 |
16 | sys.exit(serve_team())
=== END FILE: vero_status_server.py ===
```

```
=== VERIFY SCRIPT: verify_rebuild.py (48 lines) ===
```

```

1 | import hashlib, pathlib, sys
2 | # usage: py verify_rebuild.py C:\Tools\vero-status
3 | root = pathlib.Path(sys.argv[1] if len(sys.argv) > 1 else "vero-status")
4 | CRLF = ['START_HERE.txt', 'scripts/vero.cmd']
5 | FILES = {
6 |     '.gitattributes': 'efa38e63b1747daed572feeff39747d40fd4cfa3a4568b11815cb0bf24332d96',
7 |     '.gitignore': 'b592d17a523d7d8d8bd950419afdeb5f502d4aedd8b65a2887bd71f0c514aa33',
8 |     'CHANGELOG.md': '9e4bf2019070d5e3d68965891bc318e750e39894aafe938de4e57b6dc7dfb463',
9 |     'README.md': 'd2d8209f7a4be482593f1c1e91dfb14d725429ef5550a490344c5ea5b292d6de',
10 |    'START_HERE.txt': '3d6c38c0e04d3c5791716f49bab854b9ab72ed69d2824c3eccd6363e5c3682ba',
11 |    'VERO_CONTEXT.md': 'ea8d9dd774e16d3a6b25c667d8ea6e32d42bafc91566d30b28f02f7511719c0f',
12 |    'Vero Status.pyw': 'd225f8171b6196c264964f66cba8c2992695a171db6c2f047f9c694fb8358197',
13 |    'pyproject.toml': 'dcdab737202f10c2b1610fa9020563de6021071504855e756ae8d5467fd017e9',
14 |    'scripts/build_source_pdf.py': 'f3fa158ab8de632655566bc17697fcf54f028e743f0c56dccbf90bc4ccd690',
15 |    'scripts/fake_session.py': '5fa5b85d17b1e8ae07e72460a01f242d7e5223c55eb96f4d6d31fb8087a007a7',
16 |    'scripts/fake_vero.py': '4908eb33e4a64621f955e551ef0efce2e644ca3a6ba51e9c202b80cf01d35ae8',
17 |    'scripts/install_vm.sh': '23c89644435950587375e4e986ca43a702b9cbaf619849aa976e1832242e0b42',
18 |    'scripts/vero': '45890958a3fcde4e02f1d4c41412869e6f71a9efe2d2d237cd54cf8d84e001fa',
19 |    'scripts/vero.cmd': '15e9a293d205d911b2e2746af1c4361906c0b4138dc111be5acc2a653a6cd0ca',
20 |    'tests/__init__.py': '8d8ab1348161355b2c6856609575f5becac99c397071c7dcdad95d00597f39fb',
21 |    'tests/helpers.py': '1ee6057626c3d29cae5cca50a5a4583151c3c6d109d17927c066263c15523e03',
22 |    'tests/test_monitor_server.py': '60dabb972cad94f42b283bb9e7c0dc432882a58cc4644a9fde869e783c574a2a',
23 |    'tests/test_sessions.py': '86152ace8a67751301e0e94be87619f3133321a62be2381b911ec89b8e83fe36',
24 |    'tests/test_settings.py': 'd5c501ab341e42d07021d55e29832d40a0780fa8bac8fae525af14359183b836',
25 |    'tests/test_vero.py': '972fb76ca969e62456c382dc55d47f9f25ad0e5c3ccf49f85b841bd7af93d65d',
26 |    'vero_status/__init__.py': 'db5eb515f4f26125dc8f7a6a34341a0211a1db05a20286dbdebd1885e198ed81',
27 |    'vero_status/app.py': '80ce46725991064987f6f5e98a35fa38e4effa1ecf40960ac6bc404cad814c6e',
28 |    'vero_status/monitor.py': 'cd0dc809649408e8ce6e3eedea4aad60bcee9c61e9fcb1ae2935c4e99131da9',
29 |    'vero_status/server.py': 'ef08ac947eff7830d8f8611c469d6b5e4c0281fb3a095cd2927931acf2a00348',
30 |    'vero_status/sessions.py': '837e129dc53707a16169ce28fc9ce6533db4a1d82bbeab75134e3fcf57d6f03a',
31 |    'vero_status/settings.py': 'd362a34291c878bc40a6a3998a3a19633e9918aa1ac8d4352f0add2618c47d2d',
32 |    'vero_status/vero.py': '2a429351dc5a4b26352b4feb3dd6f0ba40f7e388c6cbf7b9e7ee6524ac82e4',
33 |    'vero_status/web/index.html': '355dd1b6a1b3b4bedb7c24b0c67f2541d800785b9ccaa103076832b8a0c0523c',
34 |    'vero_status_server.py': 'e05c2030adc4991a41d93f95cefd1d5d6785623dc96bf9eb10c20bf4a44f05a0',
35 | }
36 | bad = 0
37 | for path, sha in FILES.items():
38 |     p = root / path
39 |     if not p.exists():
40 |         print("MISSING", path); bad += 1; continue
41 |     text = p.read_bytes().decode("utf-8").replace("\r\n", "\n")
42 |     if hashlib.sha256(text.encode("utf-8")).hexdigest() != sha:
43 |         print("DIFFERENT", path); bad += 1; continue
44 |     want = text.replace("\n", "\r\n") if path in CRLF else text
45 |     if p.read_bytes() != want.encode("utf-8"):
46 |         p.write_bytes(want.encode("utf-8")); print("fixed line endings", path)
47 |     print("OK: all files match" if not bad else str(bad) + " file(s) wrong: recreate them from the document")
48 | sys.exit(1 if bad else 0)
=== END VERIFY SCRIPT ===
```